

Finish git flow dashboard feature branch

```
[user] tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (feature/dashboard_view)
$ git flow feature finish dashboard_view
Fetching from remote 'origin'...
Note: Could not fetch base branch 'develop': failed to fetch branch 'develop' from 'origin': fatal: couldn't find remote ref develop
Note: Could not fetch topic branch 'feature/dashboard_view': failed to fetch branch 'feature/dashboard_view' from 'origin': fatal: couldn't find remote ref feature/dashboard_view
Fetch completed
Switched to branch 'develop'
Merging using strategy: merge
Successfully finished branch 'feature/dashboard_view' and updated 0 child base branches
[assistant] Your `feature/dashboard_view` branch has been successfully merged into `develop` and cleaned up. A few things to note from the output:
```

The fetch warnings about `develop` and `feature/dashboard\_view` not being found on the remote are harmless in this case – git flow completed the merge locally just fine. It just means those branches weren't pushed to `origin` beforehand.

From here, common next steps would be:

- ****Push develop to origin**** so the remote is in sync: ``git push origin develop``
- ****Start a new feature**** if you're ready: ``git flow feature start <feature-name>``
- ****Start a release**** when develop is ready to ship: ``git flow release start <version>``

Want me to help with anything in the codebase now that the dashboard view is merged?

[user] I am not sure if I am using git flow correctly

[assistant] Looking at your screenshot, I can spot the issue – your git flow usage has a key problem.

****What happened:**** You ran ``git flow feature finish dashboard_view`` but had ****6 uncommitted changes****. The feature branch itself likely had no commits on it, so the merge was essentially empty. Your work was never committed to the feature branch before finishing it – now those changes are just floating as uncommitted on `develop`.

****The correct git flow cycle is:****

...

1. `git flow feature start dashboard_view` ← creates & switches to feature branch
 2. ... make your changes ...
 3. `git add .` ← stage changes
 4. `git commit -m "feat: add dashboard"` ← COMMIT on the feature branch ■
 5. `git flow feature finish dashboard_view` ← THEN merge into develop
- ...

You skipped step 3 and 4 – committing your work to the feature branch before finishing it.

****What you should do right now:****

Since the changes are now on `develop`, just commit them there:

```
```bash
git add .
git commit -m "feat: add dashboard view"
git push origin develop
```
```

It's not ideal (commits should ideally land on `develop` via feature merges, not directly), but it's the pragmatic fix given where things stand.

****Going forward****, think of it this way: a feature branch is useless unless you commit to it. ``git flow feature finish`` just merges whatever commits exist on the branch – if there are none, nothing meaningful gets merged.

Want me to look at the changed files to help you write a proper commit message?

```
[user] yes
[assistant] (called ToolSearch)
[assistant] (called mcp__workspace__bash)
[assistant] (called mcp__workspace__bash)
[assistant] (called mcp__workspace__bash)
[assistant] The git worktree state is a bit broken from the earlier session. Let me just read the actual changed files directly – I can see them from your screenshot.
[assistant] (called Read)
[assistant] (called Read)
[assistant] (called Read)
[assistant] (called Read)
[assistant] (called Read)
[assistant] (called Read)
[assistant] Got a clear picture now. Here's what the 6 changed files actually contain:
```

```
- **`app.html`** – Added Bulma CSS, Font Awesome, and Google Fonts to the base HTML head
- **`runes.svelte.ts`** – Extended global state with `loggedInUser` (email + name) alongside `subTitle`
- **`ListRoutes.svelte`** – New component: a Bulma card showing a route with open/delete buttons
- **`SplashScreen.svelte`** – New component: full-height hero landing page with Sign Up / Login CTAs
- **`+page.svelte` (about)** – About page with app description and icon display
- **`+page.svelte` (dashboard)** – Dashboard route scaffold with placeholders for list-routes and add-route
```

Here's a well-structured commit message you can use:

```
...
```

```
feat: add dashboard view, starter pages, and UI components
```

```
- Scaffold dashboard route with list-routes and add-route placeholders
- Add SplashScreen component with hero layout and auth CTAs (signup/login)
- Add ListRoutes component with Bulma card, open and delete actions
- Add about page with app description and icon layout
- Extend runes.svelte.ts with loggedInUser global state (email, name)
- Add Bulma CSS, Font Awesome, and Google Fonts to app.html
...
```

You can paste that directly into VS Code's commit message box, then hit ****Commit**** and ****Push****. Want me to run the commit via the terminal for you?

```
[user] I will do everything, I just need answers
```

```
[assistant] Got it! Ask away.
```

```
[user] &gt; git add -A -- .
fatal: Unable to create 'C:/Users/tiago/OneDrive/Documents/GitHub/PlaceMark-2.0/.git/index.lock': File exists.
```

Another git process seems to be running in this repository, e.g. an editor opened by 'git commit'. Please make sure all processes are terminated then try again. If it still fails, a git process may have crashed in this repository earlier:
remove the file manually to continue.

```
[assistant] That's a stale lock file – no actual git process is running, it just got left behind. Delete it with:
```

```
```bash
rm C:/Users/tiago/OneDrive/Documents/GitHub/PlaceMark-2.0/.git/index.lock
```
```

Then retry your ``git add -A -- .`` and it should work fine.

[assistant] Yes, the graph looks correct! Here's what it's showing:

The only thing left is to click **Publish Branch** to push `develop` to the remote. That will make `origin/develop` exist on GitHub and sync your new commit up.

`https://git-flow.sh/` is the one I am using.

In the diagram you shared, the pink dots (feature branch) visually **branch out** from develop and then **merge back in**, creating that fork shape. That only happens when you have actual commits on the feature branch.

```

...
develop ██████████●██████████ (merge commit)
      \
feature   ●██████●██████● (your dashboard commits here)
      /
...

```

```

...
develop  ●●●●● (all commits went straight to develop)
...

```

```

```bash
git flow feature start my-feature
... make changes ...
git add .
git commit -m "feat: my changes" ← this creates the branch line
git flow feature finish my-feature
```

```

[assistant] which part would you like me to explain step by step?

[assistant] Sure! Here's each step broken down:

```
**`git flow feature start my-feature`**
```

Creates a new branch called `feature/my-feature` off of `develop` and automatically switches you to it. This is where your isolated work lives.

```
**`# ... make changes ...`**
```

You edit your files normally in VS Code or wherever. Nothing git-related yet, just coding.

```
**`git add .`**
```

Stages all your changes – tells git "these are the files I want to include in my next commit." The dot means "everything in this folder."

```
**`git commit -m "feat: my changes"`**
```

Saves a snapshot of your staged changes to the `feature/my-feature` branch with a message describing what you did. This is the step that creates the branch line in the graph – without it, the feature branch has nothing on it.

```
**`git flow feature finish my-feature`**
```

Does three things automatically:

1. Switches back to `develop`
2. Merges `feature/my-feature` into `develop`
3. Deletes the `feature/my-feature` branch (it's no longer needed)

The result is that `develop` gets your changes, and the graph shows the fork and merge because there were actual commits on the feature branch to merge in.

[user] convert this to typescript

```
````typescript
picture: String,
createdAt: { type: Date, default: Date.now },
createdBy: {
 type: Schema.Types.ObjectId,
 ref: "User",
},
superAdmin: { type: Boolean, default: false },
````
```

[assistant] This looks like a Mongoose schema definition. Here's the TypeScript equivalent:

```
````typescript
import { Schema, model, Document, Types } from "mongoose";

interface IUser extends Document {
 picture: string;
 createdAt: Date;
 createdBy: Types.ObjectId;
 superAdmin: boolean;
}

const UserSchema = new Schema<IUser>({
 picture: { type: String },
 createdAt: { type: Date, default: Date.now },
 createdBy: { type: Schema.Types.ObjectId, ref: "User" },
 superAdmin: { type: Boolean, default: false },
});

export const User = model<IUser>("User", UserSchema);
````
```

Key changes:

- Added an `IUser` interface to type the document – this gives you autocomplete and type safety when using the model
- `String` → `string` (lowercase) in the interface, since that's TypeScript's primitive type

- ``Types.ObjectId`` is the correct TypeScript type for `ObjectId` references

If you already have a schema file and just want the interface part, let me know and I'll trim it down.

[assistant] I can see the issue – you pasted the Mongoose schema syntax directly into the TypeScript interface. An interface only holds types, not Mongoose config objects. Fix it like this:

```
``typescript
export interface User {
  firstName: string;
  lastName: string;
  email: string;
  password: string;
  _id?: string;
  picture: string;
  createdAt: Date;
  createdBy: string;
  superAdmin: boolean;
}
...

```

``createdBy`` is ``string`` here because in the interface it just represents the `ObjectId` value (which comes through as a string). The ``ref``, ``default``, and ``type`` wrappers belong in the Mongoose schema definition, not the interface.

[assistant] This looks good! But there are a couple of things to fix:

****1. Missing ``Schema`` import**** – you're using ``Schema.Types.ObjectId`` but didn't import ``Schema``:

```
``typescript
import mongoose, { Model, Schema } from "mongoose";
...

```

****2. The ``try/catch`` model pattern**** – this is a workaround for hot-reloading issues (common in Next.js). Since you're using SvelteKit, you can simplify it to just:

```
``typescript
export const UserMongoose = mongoose.model<User>("User", userSchema);
...

```

So the full clean version:

```
``typescript
import type { User } from "$lib/types/placemark-types";
import mongoose, { Model, Schema } from "mongoose";

const userSchema = new mongoose.Schema<User>({
  firstName: String,
  lastName: String,
  email: String,
  password: String,
  picture: String,
  createdAt: { type: Date, default: Date.now },
  createdBy: { type: Schema.Types.ObjectId, ref: "User" },
  superAdmin: { type: Boolean, default: false },
});

export const UserMongoose = mongoose.model<User>("User", userSchema);
...

```

[assistant] The error is because your ``User`` interface has ``createdBy: string`` but the schema is using ``Schema.Types.ObjectId``. TypeScript sees a mismatch.

Two ways to fix it:

****Option 1 – Change the interface to accept ObjectId:****

```
```typescript
import { Types } from "mongoose";

export interface User {
 ...
 createdBy: Types.ObjectId;
 ...
}
```
```

****Option 2 – Change the schema to use String (simpler):****

```
```typescript
createdBy: { type: String, ref: "User" },
```
```

I'd recommend ****Option 1**** – it's more correct since ObjectId is what Mongoose actually stores. Option 2 works but loses the ObjectId type safety.
[user] explain each part please

```
```typescript
async findBySuperAdminId(superAdminId: string): Promise<User[] | null>; {
 const users = await UserMongoose.find({ createdBy: superAdminId }).lean();
 return users;
},
```
```

[assistant] Sure!

****`async`**** – marks the function as asynchronous, meaning it can use ``await`` inside it and will return a Promise.

****`findBySuperAdminId(superAdminId: string)`**** – the function name, takes one parameter ``superAdminId`` which must be a string. This is the ID you pass in to search by.

****`Promise<User[] | null>`**** – the return type. This function will return a Promise that resolves to either an array of ``User`` objects, or ``null`` if nothing is found.

****`const users = await UserMongoose.find({ createdBy: superAdminId }).lean()`****

- ``UserMongoose.find(...)`` – queries MongoDB for all documents where ``createdBy`` matches the ``superAdminId`` you passed in
- ``await`` – waits for the database query to finish before moving on
- ``lean()`` – returns plain JavaScript objects instead of full Mongoose documents, which is lighter and faster since you don't need Mongoose methods on the result

****`return users`**** – returns the array of matching users (or an empty array ``[]`` if none were found – note that ``find()`` never actually returns ``null``, it returns an empty array, so the ``| null`` in the return type is technically unnecessary here).

[user] i added chai and mocha for testing. before it was js , should I do testing in TS now?

[assistant] Yes, since your project is already in TypeScript, you should write your tests in TypeScript too. It keeps everything consistent and gives you the same type safety in your tests as in your code – so if you pass the wrong type to a function in a test, TypeScript will catch it before you even run it.

You'll need a couple of extra packages to make chai and mocha work with TypeScript:

```
```bash
npm install --save-dev ts-node @types/chai @types/mocha
```
```

And in your ``package.json`` test script, change it to use ``ts-node``:

```
```json
"test": "mocha --require ts-node/register 'test/**/*.test.ts'"
```
```

Your test files then just become `.test.ts` instead of `.test.js`, and you can import your types directly into the tests.

```
[user] tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (feature/mongodb)
```

```
$ npm audit
```

```
# npm audit report
```

```
cookie <0.7.0
```

cookie accepts cookie name, path, and domain with out of bounds characters - <https://github.com/advisories/GHSA-pxg6-pf52-xh8x>

fix available via `npm audit fix --force`

Will install `@sveltejs/kit@0.0.30`, which is a breaking change

```
node_modules/cookie
```

```
  @sveltejs/kit >=1.0.0-next.0
```

Depends on vulnerable versions of cookie

```
node_modules/@sveltejs/kit
```

```
  @sveltejs/adapter-auto >=1.0.0-next.0
```

Depends on vulnerable versions of `@sveltejs/kit`

```
node_modules/@sveltejs/adapter-auto
```

```
diff 6.0.0 - 8.0.2
```

jsdiff has a Denial of Service vulnerability in `parsePatch` and `applyPatch` - <https://github.com/advisories/GHSA-73rr-hh4g-fpgx>

fix available via `npm audit fix --force`

Will install `mocha@11.3.0`, which is a breaking change

```
node_modules/diff
```

```
  mocha 8.0.0 - 12.0.0-beta-3
```

Depends on vulnerable versions of diff

Depends on vulnerable versions of `serialize-javascript`

```
node_modules/mocha
```

```
serialize-javascript <=7.0.4
```

Severity: high

Serialize JavaScript is Vulnerable to RCE via `RegExp.flags` and `Date.prototype.toString()` - <https://github.com/advisories/GHSA-5c6j-r48x-rmvq>

Serialize JavaScript has CPU Exhaustion Denial of Service via crafted array-like objects - <https://github.com/advisories/GHSA-qj8w-gfj5-8c6v>

fix available via `npm audit fix --force`

Will install `mocha@11.3.0`, which is a breaking change

```
node_modules/serialize-javascript
```

6 vulnerabilities (4 low, 2 high)

To address all issues (including breaking changes), run:

```
  npm audit fix --force
```

```
[assistant] **Do NOT run `npm audit fix --force` -- it would downgrade SvelteKit to `0.0.30` which would completely break your project.
```

Here's what each vulnerability actually means for you:

cookie (low) – in @sveltejs/kit

This is inside SvelteKit itself, not your code. The fix would destroy your SvelteKit version. Leave it for now and wait for SvelteKit to release a patched version.

diff + serialize-javascript (high) – in mocha

These are in your testing tool, which is a `devDependency` – it never runs in production, only locally when you run tests. The risk is much lower because of that. You can safely upgrade just mocha:

```
```bash
```

```
npm install --save-dev mocha@latest
```

```
```
```

That should fix the mocha-related vulnerabilities without touching SvelteKit.

Bottom line:

- The high severity ones are in dev tools only – not a production risk
- The SvelteKit one is low severity and not worth breaking your app over
- Keep an eye on SvelteKit releases for a patched version of `cookie`

[user] tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (feature/mongodb)

\$ npm install --save-dev mocha@latest

up to date, audited 273 packages in 2s

73 packages are looking for funding

run `npm fund` for details

6 vulnerabilities (4 low, 2 high)

To address all issues (including breaking changes), run:

npm audit fix --force

Run `npm audit` for details.

[assistant] You're already on the latest mocha – it didn't actually upgrade anything ("up to date"). This means the vulnerabilities in mocha's dependencies haven't been patched upstream yet.

Nothing you can do on your end right now. Just leave it as is – since these are `devDependencies` they pose no risk to your running app. Check back in a few weeks and run `npm audit` again to see if patches have been released.

[user] ``typescript

Help to convert this to TS?

```
import { assert } from "chai";
```

```
import { dbConnect } from "../src/lib/models/db.js";
```

```
import { tiago, testUsers } from "../fixtures.js";
```

```
import { assertSubset } from "../test-utils.js";
```

```
suite("User Model tests", () => {
```

```
  setup(async () => {
```

```
    await dbConnect();
```

```
    await dbConnect.userStore.deleteAll();
```

```
  });
```

```
  test("create a user", async () => {
```

```
    const newUser = await dbConnect.userStore.addUser(tiago);
```

```
    assertSubset(tiago, newUser);
```

```
  });
```

```
  test("get a user - success", async () => {
```

```
    const user = await db.userStore.addUser(tiago);
```

```
    const returnedUser1 = await db.userStore.getUserById(user._id);
```

```
    assert.deepEqual(user, returnedUser1);
```

```
    const returnedUser2 = await db.userStore.getUserByEmail(user.email);
```

```
    assert.deepEqual(user, returnedUser2);
```

```
  });
```

```
  test("delete One User - success", async () => {
```

```
    for (let i = 0; i < testUsers.length; i += 1) {
```

```
      // eslint-disable-next-line no-await-in-loop
```

```
      testUsers[i] = await db.userStore.addUser(testUsers[i]);
```

```
    }
```

```
    await db.userStore.deleteUserById(testUsers[0]._id);
```

```
    const returnedUsers = await db.userStore.getAllUsers();
```

```
    assert.equal(returnedUsers.length, testUsers.length - 1);
```

```
    const deletedUser = await db.userStore.getUserById(testUsers[0]._id);
```

```
    assert.isNull(deletedUser);
```

```
  });
```



```

test("delete all users", async () => {
  for (let i = 0; i < testUsers.length; i += 1) {
    // eslint-disable-next-line no-await-in-loop
    await db.userStore.addUser(testUsers[i]);
  }
  let returnedUsers = await db.userStore.getAllUsers();
  assert.equal(returnedUsers.length, 3);
  await db.userStore.deleteAll();
  returnedUsers = await db.userStore.getAllUsers();
  assert.equal(returnedUsers.length, 0);
});

test("get a user - failures", async () => {
  const noUserWithId = await db.userStore.getUserById("123");
  assert.isNull(noUserWithId);
  const noUserWithEmail = await db.userStore.getUserByEmail("no@one.com");
  assert.isNull(noUserWithEmail);
});

test("get a user - bad params", async () => {
  let nullUser = await db.userStore.getUserByEmail("");
  assert.isNull(nullUser);
  nullUser = await db.userStore.getUserById("");
  assert.isNull(nullUser);
  nullUser = await db.userStore.getUserById();
  assert.isNull(nullUser);
});

test("delete One User - fail", async () => {
  for (let i = 0; i < testUsers.length; i += 1) {
    // eslint-disable-next-line no-await-in-loop
    testUsers[i] = await db.userStore.addUser(testUsers[i]);
  }
  await db.userStore.deleteUserById("bad-id");
  const allUsers = await db.userStore.getAllUsers();
  assert.equal(testUsers.length, allUsers.length);
});
});

```

[assistant] Here's the TypeScript version, and I also spotted a bug in the original – `setup` uses `dbConnect` but all the tests use `db`. I'll fix that too:

```

```typescript
import { assert } from "chai";
import { db } from "../../src/lib/models/db.js";
import { tiago, testUsers } from "../../fixtures.js";
import { assertSubset } from "../../test-utils.js";
import type { User } from "../../src/lib/types/placemark-types.js";

suite("User Model tests", () => {

 setup(async () => {
 await db.connect();
 await db.userStore.deleteAll();
 });

```

```

test("create a user", async () => {
 const newUser: User = await db.userStore.addUser(tiago);
 assertSubset(tiago, newUser);
});

test("get a user - success", async () => {
 const user: User = await db.userStore.addUser(tiago);
 const returnedUser1: User | null = await db.userStore.getUserById(user._id);
 assert.deepEqual(user, returnedUser1);
 const returnedUser2: User | null = await db.userStore.getUserByEmail(user.email);
 assert.deepEqual(user, returnedUser2);
});

test("delete One User - success", async () => {
 for (let i = 0; i < testUsers.length; i += 1) {
 testUsers[i] = await db.userStore.addUser(testUsers[i]);
 }
 await db.userStore.deleteUserById(testUsers[0]._id);
 const returnedUsers: User[] = await db.userStore.getAllUsers();
 assert.equal(returnedUsers.length, testUsers.length - 1);
 const deletedUser: User | null = await db.userStore.getUserById(testUsers[0]._id);
 assert.isNull(deletedUser);
});

test("delete all users", async () => {
 for (let i = 0; i < testUsers.length; i += 1) {
 await db.userStore.addUser(testUsers[i]);
 }
 let returnedUsers: User[] = await db.userStore.getAllUsers();
 assert.equal(returnedUsers.length, 3);
 await db.userStore.deleteAll();
 returnedUsers = await db.userStore.getAllUsers();
 assert.equal(returnedUsers.length, 0);
});

test("get a user - failures", async () => {
 const noUserWithId: User | null = await db.userStore.getUserById("123");
 assert.isNull(noUserWithId);
 const noUserWithEmail: User | null = await db.userStore.getUserByEmail("no@one.com");
 assert.isNull(noUserWithEmail);
});

test("get a user - bad params", async () => {
 let nullUser: User | null = await db.userStore.getUserByEmail("");
 assert.isNull(nullUser);
 nullUser = await db.userStore.getUserById("");
 assert.isNull(nullUser);
 nullUser = await db.userStore.getUserById(undefined);
 assert.isNull(nullUser);
});

test("delete One User - fail", async () => {
 for (let i = 0; i < testUsers.length; i += 1) {
 testUsers[i] = await db.userStore.addUser(testUsers[i]);
 }
 await db.userStore.deleteUserById("bad-id");

```

```

 const allUsers: User[] = await db.userStore.getAllUsers();
 assert.equal(testUsers.length, allUsers.length);
 });
});

```

Key changes made:

- Fixed `dbConnect` → `db` consistently across all tests (this was a bug in the original)
- Added `User` and `User | null` types to all variables holding user data
- Changed `getUserById()` with no args to `getUserById(undefined)` – TypeScript won't allow calling it with no argument unless the parameter is optional in your store definition
- Removed `eslint-disable` comments – not needed in TS

Make sure your `fixtures.ts` and `test-utils.ts` files are also converted to TypeScript, otherwise the imports will cause issues.  
[assistant] TypeScript doesn't know about mocha's globals like `suite`, `setup`, and `test`. Two things to fix:

```

1. Install the types:
```bash
npm install --save-dev @types/mocha
```

```

```

2. Add mocha to your `tsconfig.json`:
```json
{
  "compilerOptions": {
    "types": ["mocha"]
  }
}
```

```

That tells TypeScript to include mocha's type definitions globally, so `suite`, `test`, `setup` etc. are all recognised without needing to import them.

[user] <https://tutors.dev/lab/setu-hdip-comp-sci-2025-full-stack-1/unit-6-sveltekit/topic-23-svelte-full/side/book-23a/03#src%2Fhooks.server.ts>

```

...
import { dbConnect } from "$lib/models/db";
import type { Handle } from "@sveltejs/kit";

await dbConnect();

export const handle: Handle = async ({ event, resolve }) => {
 const response = await resolve(event);
 return response;
};
...

```

This is a hook function - that will run on the server when the application launches.

If you restart the app you should see a successful connection message (on the server log), if you have the database running locally.

My data base is not locally, how to I know this hooks.server.ts is being called

[assistant] A few easy ways to verify it's being called:

**\*\*1. Add a console.log\*\*** – the simplest way:

```

```typescript
import { dbConnect } from "$lib/models/db";
import type { Handle } from "@sveltejs/kit";

await dbConnect();
console.log("hooks.server.ts was called ✓");

export const handle: Handle = async ({ event, resolve }) => {
  const response = await resolve(event);
  return response;
};
```

```

When you run `npm run dev`, check your **terminal** (not the browser console) – you'll see the log there since this runs on the server side.

**\*\*2. Your `dbConnect` function should already log\*\*** – when the connection to MongoDB Atlas succeeds, Mongoose fires a connection event. Make sure your `db.ts` has something like:

```

```typescript
mongoose.connection.on("connected", () => {
  console.log("Connected to MongoDB");
});
```

```

**\*\*3. Check MongoDB Atlas directly\*\*** – go to your Atlas dashboard → your cluster → **Monitoring** tab. If the hook ran and connected, you'll see incoming connections there.

The key thing to remember is that `hooks.server.ts` logs always appear in the **terminal**, never in the browser console.

[user] where in the project is it being initiated? or is it automatic in svelte kit ?

[assistant] It's **automatic in SvelteKit**. You don't need to call or import it anywhere.

SvelteKit looks for a file specifically named `hooks.server.ts` (or `hooks.server.js`) in your `src` folder and runs it automatically when the server starts. It's a convention SvelteKit follows, similar to how it looks for `+page.svelte` or `+layout.svelte` – the filename itself is what gives it special meaning.

So your project structure just needs:

```

...
src/
 hooks.server.ts ← SvelteKit picks this up automatically
 routes/
 ...
 lib/
 ...
...

```

No imports, no registration – just having the file in the right place is enough.

[user] tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (feature/mongodb)

\$ npm run test

> placemark-2.0@0.0.1 test

> mocha --require ts-node/register 'test/\*\*/\*.test.ts'

Error: No test files found: "test/\*\*/\*.test.ts"

[assistant] The quotes around the glob pattern are causing the issue on Windows. Try:

```

```bash
mocha --require ts-node/register test/**/*.test.ts
```

```

```

If that still doesn't work, update your `package.json` script to use the `--spec` flag:

```
```json
"test": "mocha --require ts-node/register --spec test/**/*.test.ts"
```
```

Or specify the file directly to confirm mocha itself is working:

```
```bash
mocha --require ts-node/register test/models/user.test.ts
```
```

Windows handles glob patterns differently from Linux/Mac, which is why the quotes break it.

[assistant] Your test folder structure uses `model` (singular) not `models`, so update the path in your script to match:

```
```bash
mocha --require ts-node/register test/model/user-model-test.ts
```
```

And update `package.json`:

```
```json
"test": "mocha --require ts-node/register test/model/user-model-test.ts"
```
```

```
[user] ```json
      "test": "mocha --require ts-node/register 'test/**/*.ts'"
```
```

is it correct? as there will be more files later on

[assistant] On Windows, remove the quotes around the glob:

```
```json
"test": "mocha --require ts-node/register test/**/*.ts"
```
```

This will automatically pick up all `.ts` files in any subfolder under `test/` as you add more test files later.

[user] comments in TS?

[assistant] Same as JavaScript:

```
```typescript
// single line comment
```

```
/* multi line
   comment */
```

```
/** JSDoc comment - used for documenting functions
```

```
 * @param id - the user id
 * @returns the user object
 */
```
```

[user] ```typescript

```
 setup(async () => {
 await dbConnect();
```

```

 await dbConnect.userStore.deleteAll();
 });

 test("create a user", async () => {
 const newUser = await dbConnect.userStore.addUser(tiago);
 assertSubset(tiago, newUser);

 });
}

```

is it correct ?

[assistant] No – `dbConnect` is a **function**, not an object with a `userStore` property. You can't call `dbConnect.userStore.deleteAll()`.

It should be:

```

````typescript
setup(async () => {
  await db.connect();
  await db.userStore.deleteAll();
});

test("create a user", async () => {
  const newUser = await db.userStore.addUser(tiago);
  assertSubset(tiago, newUser);
});
````

```

Where `db` is imported from your `db.ts` file. Can you share what your `db.ts` exports? That way I can confirm the correct import and usage.

```

[user] ````typescript
import mongoose from "mongoose";
import { MONGO_URL } from "$env/static/private";
/*
 0 - disconnected
 1 - connected
 2 - connecting
 3 - disconnecting
 4 - uninitialized
*/
const mongoConnection = {
 isConnected: 0
};

export const dbConnect = async () => {
 console.log("MONGO_URL", MONGO_URL);
 if (mongoConnection.isConnected === 1) {
 console.log("already connected");
 return;
 }

 if (mongoose.connections.length > 0) {
 mongoConnection.isConnected = mongoose.connections[0].readyState;
 if (mongoConnection.isConnected === 1) {
 console.log("using existing connection");
 return;
 }
 }
}

```

```

 await mongoose.disconnect();
 }
 await mongoose.connect(MONGO_URL);
 mongoConnection.isConnected = 1;
 console.log("conected to ", MONGO_URL ?? "");
};

export const dbDisconnect = async () => {
 if (process.env.NODE_ENV === "development") return;
 if (mongoConnection.isConnected === 0) return;

 await mongoose.disconnect();
 mongoConnection.isConnected = 0;
 console.log("disconnecting");
};

...

```

[assistant] I see the issue – `db.ts` only exports `dbConnect` and `dbDisconnect`. There's no `userStore` attached to it.

Your test setup should be:

```

````typescript
import { dbConnect } from "../../src/lib/models/db.js";
import { userStore } from "../../src/lib/models/mongo/user-store.js"; // import directly

setup(async () => {
  await dbConnect();
  await userStore.deleteAll();
});

test("create a user", async () => {
  const newUser = await userStore.addUser(tiago);
  assertSubset(tiago, newUser);
});
````

```

Import `userStore` directly from your user store file rather than trying to access it through `db`. The `db.ts` file is only responsible for managing the connection, not exposing the stores.

[assistant] I can see from the tooltip that your `userStore` has these methods:

- `find()`, `findOne()`, `add()`, `findByEmail()`, `findBySuperAdminId()`, `deleteOne()`, `delete()`, `updateOne()`, `updatePassword()`

These names are different from what the tests are calling (`addUser`, `deleteAll`, `getUserById`, etc.). You need to match the test calls to the actual method names:

| Test is calling                   | Should be                      |
|-----------------------------------|--------------------------------|
| `userStore.deleteAll()`           | `userStore.delete()`           |
| `userStore.addUser(tiago)`        | `userStore.add(tiago)`         |
| `userStore.getUserById(id)`       | `userStore.findOne(id)`        |
| `userStore.getUserByEmail(email)` | `userStore.findByEmail(email)` |
| `userStore.getAllUsers()`         | `userStore.find()`             |
| `userStore.deleteUserById(id)`    | `userStore.deleteOne(id)`      |

Also the error `Property 'userStore' does not exist on type '() => Promise<void>` means you're still calling `dbConnect.userStore` somewhere – `dbConnect` is just a function, import `userStore` directly.

[user] I need this in typescript. but write it from scratch and show me.

```
```typescript
import { assert } from "chai";
import { tiago, testUsers } from "../fixtures.js";
import { assertSubset } from "../test-utils.js";
import { userStore } from "../../src/lib/models/mongo/user-store.js";

suite("User Model tests", () => {

  setup(async () => {
    await userStore.delete();
  });

  test("create a user", async () => {
    const newUser = await userStore.addUser(tiago);
    assertSubset(tiago, newUser);
  });
});

[assistant] ```typescript
import { assert } from "chai";
import { dbConnect } from "../../src/lib/models/db.js";
import { userStore } from "../../src/lib/models/mongo/user-store.js";
import { tiago, testUsers } from "../fixtures.js";
import { assertSubset } from "../test-utils.js";
import type { User } from "../../src/lib/types/placemark-types.js";

suite("User Model tests", () => {

  setup(async () => {
    await dbConnect();
    await userStore.delete();
  });

  test("create a user", async () => {
    const newUser: User = await userStore.add(tiago);
    assertSubset(tiago, newUser);
  });

  test("get a user - success", async () => {
    const user: User = await userStore.add(tiago);
    const returnedUser1: User = await userStore.findOne(user._id);
    assert.deepEqual(user, returnedUser1);
    const returnedUser2: User = await userStore.findByEmail(user.email);
    assert.deepEqual(user, returnedUser2);
  });

  test("delete one user - success", async () => {
    for (let i = 0; i < testUsers.length; i += 1) {
      testUsers[i] = await userStore.add(testUsers[i]);
    }
    await userStore.deleteOne(testUsers[0]._id);
    const remainingUsers: User[] = await userStore.find();
  });
});
```



```

    assert.equal(remainingUsers.length, testUsers.length - 1);
    const deletedUser: User = await userStore.findOne(testUsers[0]._id);
    assert.isNull(deletedUser);
  });

  test("delete all users", async () => {
    for (let i = 0; i < testUsers.length; i += 1) {
      await userStore.add(testUsers[i]);
    }
    let allUsers: User[] = await userStore.find();
    assert.equal(allUsers.length, testUsers.length);
    await userStore.delete();
    allUsers = await userStore.find();
    assert.equal(allUsers.length, 0);
  });

  test("get a user - not found", async () => {
    const noUserWithId: User = await userStore.findOne("bad-id");
    assert.isNull(noUserWithId);
    const noUserWithEmail: User = await userStore.findByEmail("no@one.com");
    assert.isNull(noUserWithEmail);
  });

  test("delete one user - fail", async () => {
    for (let i = 0; i < testUsers.length; i += 1) {
      testUsers[i] = await userStore.add(testUsers[i]);
    }
    await userStore.deleteOne("bad-id");
    const allUsers: User[] = await userStore.find();
    assert.equal(allUsers.length, testUsers.length);
  });
});

```

Key things to note:

- `dbConnect()` is called in `setup` to ensure the DB is connected before each test
 - All method names match your actual `userStore` (`add`, `find`, `findOne`, `findByEmail`, `deleteOne`, `delete`)
 - Types are applied to all variables
 - You'll still need `fixtures.ts` and `test-utils.ts` converted to TypeScript
- [user] do I need to change anything in this one ?

```

```typescript
import { create } from "domain";

export const serviceUrl = "http://localhost:3000";

export const tiago = {
 firstName: "Tiago",
 lastName: "Villaca",
 email: "tiagovillaca15@gmail.com",
 password: "123456",
};

export const maggie = {
 firstName: "Maggie",

```

```
 lastName: "Simpson",
 email: "maggie@simpson.com",
 password: "secret",
 };

export const tiagoCredentials = {
 email: "tiagovillaca15@gmail.com",
 password: "123456"
};

export const testUsers = [
 {
 firstName: "Homer",
 lastName: "Simpson",
 email: "homer@simpson.com",
 password: "secret",
 },
 {
 firstName: "Marge",
 lastName: "Simpson",
 email: "marge@simpson.com",
 password: "secret",
 },
 {
 firstName: "Bart",
 lastName: "Simpson",
 email: "bart@simpson.com",
 password: "secret",
 },
];

export const testSingleRoute = {
 title: "Route 6 - Donegal to Dublin",
 isPublicRoute: true,
};

export const testRoutes = [
 {
 title: "Route 1 - Galway to Dublin",
 isPublicRoute: true,
 },
 {
 title: "Route 2 - Dublin to Cork",
 isPublicRoute: true,
 },
 {
 title: "Route 3 - Limerick to Waterford",
 isPublicRoute: true,
 },
 {
 title: "Route 4 - Sligo to Athlone",
 isPublicRoute: true,
 },
 {
 title: "Route 5 - Belfast to Dublin",
 isPublicRoute: true,
 },
];
```

```

 },
];

export const stop1 = {
 stopNumber: "ST00113",
 description: "Galway Bus Station",
 latitude: 53.2744,
 longitude: -9.0489,
 stopType: "Bus",
 isPublicStop: true,
};

export const testStops = [
 {
 stopNumber: "ST00112",
 description: "Galway Bus Station",
 latitude: 53.2744,
 longitude: -9.0489,
 stopType: "Bus",
 isPublicStop: true,
 },
 {
 stopNumber: "ST00212",
 description: "Heuston Station",
 latitude: 53.3463,
 longitude: -6.2927,
 stopType: "Train",
 isPublicStop: true,
 },
 {
 stopNumber: "ST00312",
 description: "Cork City Centre",
 latitude: 51.8985,
 longitude: -8.4756,
 stopType: "Bus",
 isPublicStop: false,
 },
 {
 stopNumber: "ST00412",
 description: "Waterford Ferry Terminal",
 latitude: 52.2593,
 longitude: -7.1101,
 stopType: "Ferry",
 isPublicStop: true,
 },
 {
 stopNumber: "ST00512",
 description: "Dublin Metro Point",
 latitude: 53.3498,
 longitude: -6.2603,
 stopType: "Metro",
 isPublicStop: false,
 },
];

```

[assistant] Yes, a few things to fix:

```
````typescript
import type { User } from "../src/lib/types/placemark-types.js";

export const serviceUrl = "http://localhost:3000";

export const tiago: User = {
  firstName: "Tiago",
  lastName: "Villaca",
  email: "tiagovillaca15@gmail.com",
  password: "123456",
};

export const maggie: User = {
  firstName: "Maggie",
  lastName: "Simpson",
  email: "maggie@simpson.com",
  password: "secret",
};

export const tiagoCredentials = {
  email: "tiagovillaca15@gmail.com",
  password: "123456",
};

export const testUsers: User[] = [
  {
    firstName: "Homer",
    lastName: "Simpson",
    email: "homer@simpson.com",
    password: "secret",
  },
  {
    firstName: "Marge",
    lastName: "Simpson",
    email: "marge@simpson.com",
    password: "secret",
  },
  {
    firstName: "Bart",
    lastName: "Simpson",
    email: "bart@simpson.com",
    password: "secret",
  },
];

export const testSingleRoute = {
  title: "Route 6 - Donegal to Dublin",
  isPublicRoute: true,
};

export const testRoutes = [
  { title: "Route 1 - Galway to Dublin", isPublicRoute: true },
  { title: "Route 2 - Dublin to Cork", isPublicRoute: true },
  { title: "Route 3 - Limerick to Waterford", isPublicRoute: true },
  { title: "Route 4 - Sligo to Athlone", isPublicRoute: true },
  { title: "Route 5 - Belfast to Dublin", isPublicRoute: true },
];
```

```

];

export const stop1 = {
  stopNumber: "ST00113",
  description: "Galway Bus Station",
  latitude: 53.2744,
  longitude: -9.0489,
  stopType: "Bus",
  isPublicStop: true,
};

export const testStops = [
  { stopNumber: "ST00112", description: "Galway Bus Station", latitude: 53.2744, longitude: -9.0489, stopType: "Bus", isPublicStop: true },
  { stopNumber: "ST00212", description: "Heuston Station", latitude: 53.3463, longitude: -6.2927, stopType: "Train", isPublicStop: true },
  { stopNumber: "ST00312", description: "Cork City Centre", latitude: 51.8985, longitude: -8.4756, stopType: "Bus", isPublicStop: false },
  { stopNumber: "ST00412", description: "Waterford Ferry Terminal", latitude: 52.2593, longitude: -7.1101, stopType: "Ferry", isPublicStop: true },
  { stopNumber: "ST00512", description: "Dublin Metro Point", latitude: 53.3498, longitude: -6.2603, stopType: "Metro", isPublicStop: false },
];

```

Changes made:

- Removed ``import { create } from "domain"`` – this was unused and shouldn't be there
- Added ``import type { User }`` from your types file
- Typed ``tiago``, ``maggie`` as ``User`` and ``testUsers`` as ``User[]``
- Added missing ``const`` on ``testUsers`` (your original had ``export testUsers`` which is a syntax error)

[user] Exception during run: Error [ERR_REQUIRE_ESM]: Must use import to load ES Module: C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\test\fixtures.ts
require() of ES modules is not supported.

require() of C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\test\fixtures.ts from C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\mocha\lib\nodejs\esm-utils.js is an ES module file as it is a .ts file whose nearest parent package.json contains "type": "module" which defines all .ts files in that package scope as ES modules.

Instead change the requiring code to use import(), or remove "type": "module" from C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\package.json.

```

    at createErrRequireEsm (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\ts-node\dist-raw\node-internal-errors.js:46:15)
    at assertScriptCanLoadAsCJSImpl (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\ts-node\dist-raw\node-internal-modules-cjs-loader.js:584:11)
    at Object.require.extensions.&lt;computed> [as .ts] (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\ts-node\src\index.ts:1610:5)
    at Module.load (node:internal/modules/cjs/loader:1289:32)
    at Function._load (node:internal/modules/cjs/loader:1108:12)
    at TracingChannel.traceSync (node:diagnostics_channel:322:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:220:24)
    at Module.require (node:internal/modules/cjs/loader:1311:12)
    at require (node:internal/modules/helpers:136:16)
    at Object.requireModule [as requireOrImport] (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\mocha\lib\nodejs\esm-utils.js:98:12)
    at Object.exports.loadFilesAsync (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\mocha\lib\nodejs\esm-utils.js:152:34)
    at Mocha.loadFilesAsync (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\mocha\lib\mocha.js:429:19)
    at singleRun (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\mocha\lib\cli\run-helpers.js:168:15)
    at exports.runMocha (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\mocha\lib\cli\run-helpers.js:245:10)
    at Object.exports.handler (C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\mocha\lib\cli\run.js:379:11)
    at C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\yargs\build\index.cjs:1:8993
    at C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\node_modules\yargs\build\index.cjs:1:4949 {
  code: 'ERR_REQUIRE_ESM'
}

```

[assistant] The problem is your ``package.json`` has ``"type": "module"`` which means everything is ESM, but ``ts-node`` is trying to use CommonJS

```
`require()`.`
```

Update your test script in `package.json` to use `ts-node/esm` instead:

```
```json
"test": "mocha --loader ts-node/esm test/**/*.ts"
```
```

And add a `.mocharc.cjs` file in your project root:

```
```js
module.exports = {
 loader: "ts-node/esm",
 spec: "test/**/*.ts",
 timeout: 10000
};
```
```

Then simplify your test script to just:

```
```json
"test": "mocha"
```
```

Mocha will pick up the `.mocharc.cjs` config automatically. The `.cjs` extension on the mocha config is important – it forces that file to be treated as CommonJS even though your project is ESM.

[user] what are the other ways?

[assistant] There are 3 other approaches:

****1. Remove `"type": "module"` from `package.json`****

The simplest fix but may break SvelteKit since it expects ESM.

****2. Use `tsx` instead of `ts-node`****

```
```bash
npm install --save-dev tsx
```

```json
"test": "mocha --loader tsx test/**/*.ts"
```

`tsx` handles ESM better than `ts-node`.
```

****3. Compile TypeScript first, then run the compiled JS****

```
```json
"pretest": "tsc --outDir dist/test",
"test": "mocha dist/test/**/*.js"
```
```

Slower workflow but avoids the ESM/CJS conflict entirely.

For your setup I'd recommend ****option 2 (tsx)**** – it's the least friction with SvelteKit's ESM requirement and simpler than configuring `ts-node` for ESM.

[user] tsx then

[assistant] ```bash

```
npm install --save-dev tsx
```
```

Then update `package.json`:

```
```json
"test": "mocha --loader tsx test/**/*.ts"
```
```

[user] there is no need for ts-node?

[assistant] Correct, you can remove it:

```
```bash
npm uninstall ts-node
```
```

`tsx` replaces `ts-node` entirely – it does the same job but handles ESM properly without extra configuration.

[user] node:internal/modules/run\_main:122

```
 triggerUncaughtException(
 ^
```

Error [ERR\_UNSUPPORTED\_ESM\_URL\_SCHEME]: Only URLs with a scheme in: file, data, and node are supported by the default ESM loader. On Windows, absolute paths must be valid file:// URLs. Received protocol 'c:'

```
 at throwIfUnsupportedURLScheme (node:internal/modules/esm/load:209:11)
 at defaultLoad (node:internal/modules/esm/load:107:3)
 at nextLoad (node:internal/modules/esm/hooks:748:28)
 at Hooks.load (node:internal/modules/esm/hooks:385:26)
 at ModuleLoader.load (node:internal/modules/esm/loader:697:35)
 at ModuleLoader.loadAndTranslate (node:internal/modules/esm/loader:514:43)
 at #createModuleJob (node:internal/modules/esm/loader:538:36)
 at #getJobFromResolveResult (node:internal/modules/esm/loader:306:34)
 at ModuleLoader.getModuleJobForImport (node:internal/modules/esm/loader:274:41)
 at process.processTicksAndRejections (node:internal/process/task_queues:105:5) {
 code: 'ERR_UNSUPPORTED_ESM_URL_SCHEME'
}
```

Node.js v22.14.0

[assistant] This is a Windows path issue – Node's ESM loader doesn't like `C:` style paths. Try this in `package.json`:

```
```json
"test": "mocha --loader tsx --experimental-specifier-resolution=node test/**/*.ts"
```
```

If that doesn't work, try using `tsx` directly instead of as a loader:

```
```json
"test": "tsx node_modules/mocha/bin/mocha.js test/**/*.ts"
```
```

Or use the `mocha` + `tsx` recommended approach for Windows:

```
```json
"test": "cross-env NODE_OPTIONS=\"--loader tsx\" mocha test/**/*.ts"
```
```

Which requires:

```
```bash
npm install --save-dev cross-env
```
```

`cross-env` is a useful package anyway for Windows development as it handles environment variable differences between Windows and Mac/Linux.

[user] &gt; tsx node\_modules/mocha/bin/mocha.js test/\*\*/\*.ts

```
Exception during run: Error [ERR_MODULE_NOT_FOUND]: Cannot find module 'C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\test\test\
fixtures.js' imported from C:\Users\tiago\OneDrive\Documents\GitHub\PlaceMark-2.0\test\model\user-model-test.ts
 at finalizeResolution (node:internal/modules/esm/resolve:275:11)
 at moduleResolve (node:internal/modules/esm/resolve:860:10)
 at defaultResolve (node:internal/modules/esm/resolve:984:11)
 at nextResolve (node:internal/modules/esm/hooks:748:28)
 at resolveBase (file:///C:/Users/tiago/OneDrive/Documents/GitHub/PlaceMark-2.0/node_modules/tsx/dist/esm/index.mjs?1778397702997:2:3744)
 at async resolveDirectory (file:///C:/Users/tiago/OneDrive/Documents/GitHub/PlaceMark-2.0/node_modules/tsx/dist/esm/
index.mjs?1778397702997:2:4237)
 at async resolve (file:///C:/Users/tiago/OneDrive/Documents/GitHub/PlaceMark-2.0/node_modules/tsx/dist/esm/index.mjs?1778397702997:2:5355)
 at async nextResolve (node:internal/modules/esm/hooks:748:22)
 at async Hooks.resolve (node:internal/modules/esm/hooks:240:24)
 at async handleMessage (node:internal/modules/esm/worker:199:18) {
 code: 'ERR_MODULE_NOT_FOUND',
 url: 'file:///C:/Users/tiago/OneDrive/Documents/GitHub/PlaceMark-2.0/test/test/fixtures.js'
}
[assistant] Two issues:
```

**\*\*1. Double `test/test/` path\*\*** – your import in `user-model-test.ts` is wrong. Change:

```
```typescript
import { tiago, testUsers } from "../fixtures.js";
```

to:
```typescript
import { tiago, testUsers } from "../fixtures.js";
```
```

Wait – looking at the error more carefully, it's looking for `test/test/fixtures.js` which means your import path has an extra `../`. Check your import in `user-model-test.ts`, it probably looks like:

```
```typescript
import { tiago, testUsers } from "../../test/fixtures.js"; // wrong
```
```

It should be:

```
```typescript
import { tiago, testUsers } from "../fixtures.js"; // correct
```
```

**\*\*2. `.js` extension on `.ts` files\*\*** – in ESM TypeScript you import `.ts` files using `.js` extension (TypeScript resolves it). So `../fixtures.js` is actually correct for `../fixtures.ts`. Just fix the path.

Share the imports at the top of `user-model-test.ts` and I can confirm exactly what needs changing.

```
[user] ```typescript
import { assert } from "chai";
import { tiago, testUsers } from "../test/fixtures.js";
import { dbConnect } from "../../src/lib/db.js";
import { assertSubset } from "../test/test-utils.js";
import { userStore } from "../../src/lib/models/mongo/user-store.js";
import type { User } from "../../src/lib/types/placemark-types.js";
```
```

[assistant] There's your problem – the paths are wrong. Your test file is at `test/model/user-model-test.ts`, so:

```
```typescript
import { assert } from "chai";
import { tiago, testUsers } from "../fixtures.js";
```
```



```
import { dbConnect } from "../../src/lib/models/db.js";
import { assertSubset } from "../test-utils.js";
import { userStore } from "../../src/lib/models/mongo/user-store.js";
import type { User } from "../../src/lib/types/placemark-types.js";
```

```

Changes:

- `../test/fixtures.js` → `../fixtures.js` (fixtures is in `test/`, not `test/test/`)
- `../test/test-utils.js` → `../test-utils.js` (same reason)
- `../../src/lib/db.js` → `../../src/lib/models/db.js` (based on your earlier file structure)

[user] and now ? path and folders are shown

[assistant] Looking at your folder structure, the imports should use `.ts` not `.js` since you're using `tsx`. Also `db.ts` is directly in `src/lib/models/`, not `src/lib/`. Fix your imports to:

```
```typescript
import { assert } from "chai";
import { tiago, testUsers } from "../fixtures.ts";
import { dbConnect } from "../../src/lib/models/db.ts";
import { assertSubset } from "../test-utils.ts";
import { userStore } from "../../src/lib/models/mongo/user-store.ts";
import type { User } from "../../src/lib/types/placemark-types.ts";
```

```

[user] mocha and chai for testing is only JS?

[assistant] No, mocha and chai work with TypeScript too – that's exactly what you're setting up right now. They're just testing libraries, the language doesn't matter as long as you have something to transpile the TypeScript (which is what `tsx` is doing for you).

[user] test folder goes inside src or outside?

[assistant] **Outside** `src`. Test files are not part of your application code, they're separate tooling. Your current structure is already correct:

...

PlaceMark-2.0/

```

■■■ src/
■ ■■■ lib/
■ ■■■ ...
■■■ test/
■ ■■■ model/
■ ■ ■■■ user-model-test.ts
■ ■■■ fixtures.ts
■ ■■■ test-utils.ts
...

```

[user] how to ignore this error ?

[assistant] Your `tsconfig.json` has `rootDir` set to `src`, which doesn't include the `test` folder. Open your `tsconfig.json` and either:

**Option 1 – Remove `rootDir`:**

```
```json
{
  "compilerOptions": {
    // remove or comment out rootDir
  }
}
```

```

**Option 2 – Change `rootDir` to project root:**

```
```json
{
  "compilerOptions": {
    "rootDir": "."
  }
}
```

```

```
}
}
...
```

Share your `tsconfig.json` and I can tell you exactly what to change.

[user] what difference does it make?

[assistant] **`rootDir: "src"`** – TypeScript only expects source files inside `src/`. Anything outside (like your `test/` folder) causes an error because it's outside the boundary.

**`rootDir: "."`** – TypeScript treats the whole project root as the source boundary, so both `src/` and `test/` are valid locations for `.ts` files.

**No `rootDir`** – TypeScript figures it out automatically based on where your `.ts` files actually are.

For a project with tests outside `src/`, either option 2 or removing it entirely is the right call. Removing it is simpler and lets TypeScript manage it automatically.

[user] tests are not showing here. how do i fix it ?

[assistant] This is VS Code's built-in Test Explorer – it doesn't know about Mocha by default. You need to install the Mocha extension for it.

Click **"Install Additional Test Extensions"** and search for **Mocha Test Explorer**, or install it directly:

1. Go to Extensions (`Ctrl+Shift+X`)
2. Search **"Mocha Test Explorer"**
3. Install it

After installing, it should automatically pick up your test files and display them in the Testing panel.

[user] it is installed already

[assistant] Then you need a `.mocharc.cjs` file in your project root to tell the extension where your tests are:

```
```js  
module.exports = {  
  loader: "tsx",  
  spec: "test/**/*.ts",  
  timeout: 10000  
};  
```
```

After adding it, reload VS Code (`Ctrl+Shift+P` → "Reload Window") and the tests should appear in the Testing panel.

[user] i didnt have this in my other project

[assistant] That's because your other project was likely JavaScript with a simpler setup. The `.mocharc.cjs` is needed here because:

1. Your project uses `"type": "module"` (ESM)
2. You're using TypeScript with `tsx` as the loader

In a plain JS project, Mocha finds tests automatically with no extra config. The more complex your setup, the more you need to tell Mocha where to look and how to handle the files.

[user] if I change the type? is there any other?

[assistant] Removing `"type": "module"` from `package.json` would likely **break SvelteKit** since it requires ESM to run. So that's not a safe option.

The other alternatives are:

**`.mocharc.yml`** instead of **`.mocharc.cjs`**:

```
```yaml  
loader: tsx  
spec: test/**/*.ts  
timeout: 10000
```

```
...
```

```
**`.mocharc.json`**:  
```json  
{
 "loader": "tsx",
 "spec": "test/**/*.ts",
 "timeout": 10000
}
```
```

All three do the same thing, just different formats. The `.cjs` version is the safest with ESM projects. Pick whichever you prefer.

```
[user] SyntaxError: Unexpected token '{'  
    at compileSourceTextModule (node:internal/modules/esm/utils:338:16)  
    at ModuleLoader.importSyncForRequire (node:internal/modules/esm/loader:353:18)  
    at loadESMFromCJS (node:internal/modules/cjs/loader:1385:24)  
    at Module._compile (node:internal/modules/cjs/loader:1536:5)  
    at Object..js (node:internal/modules/cjs/loader:1706:10)  
    at Module.load (node:internal/modules/cjs/loader:1289:32)  
    at Function._load (node:internal/modules/cjs/loader:1108:12)  
    at TracingChannel.traceSync (node:diagnostics_channel:322:14)  
    at wrapModuleLoad (node:internal/modules/cjs/loader:220:24)  
    at Module.require (node:internal/modules/cjs/loader:1311:12)
```

Stacktrace:

```
    at S.syncTopLevel (c:\Users\tiago\.vscode\extensions\ms-vscode.test-adapter-converter-0.2.1\out\extension.js:6:3)  
    at up.value (c:\Users\tiago\.vscode\extensions\ms-vscode.test-adapter-converter-0.2.1\out\extension.js:3:4541)  
    at E._deliver (file:///c:/Users/tiago/AppData/Local/Programs/Microsoft%20VS%20Code/8b640eef5a/resources/app/out/vs/workbench/api/node/  
extensionHostProcess.js:403:3244)  
    at E.fire (file:///c:/Users/tiago/AppData/Local/Programs/Microsoft%20VS%20Code/8b640eef5a/resources/app/out/vs/workbench/api/node/  
extensionHostProcess.js:403:3562)  
    at up.value (c:\Users\tiago\.vscode\extensions\hbenl.vscode-test-explorer-2.22.1\out\hub\testHub.js:53:43)  
    at E._deliver (file:///c:/Users/tiago/AppData/Local/Programs/Microsoft%20VS%20Code/8b640eef5a/resources/app/out/vs/workbench/api/node/  
extensionHostProcess.js:403:3244)  
    at E.fire (file:///c:/Users/tiago/AppData/Local/Programs/Microsoft%20VS%20Code/8b640eef5a/resources/app/out/vs/workbench/api/node/  
extensionHostProcess.js:403:3562)  
    at ChildProcess.handler (c:\Users\tiago\.vscode\extensions\hbenl.vscode-mocha-test-adapter-2.14.4\out\core.js:91:55)  
    at ChildProcess.emit (node:events:519:28)  
    at emit (node:internal/child_process:949:14)  
    at process.processTicksAndRejections (node:internal/process/task_queues:90:21)  
[assistant] The VS Code Mocha extension is having trouble with the ESM setup. This is a known compatibility issue with that extension and ESM  
projects.
```

The simplest fix – just run your tests from the **terminal** instead of the VS Code Testing panel:

```
```bash  
npm run test
```
```

The Testing panel in VS Code is just a visual convenience. Your tests will work fine from the terminal, which is the standard way to run tests in ESM/TypeScript projects anyway.

```
[user] node:internal/modules/run_main:122  
    triggerUncaughtException(  
      ^  
    )  
    
```

Error: tsx must be loaded with --import instead of --loader

```
The --loader flag was deprecated in Node v20.6.0 and v18.19.0
  at initialize (file:///C:/Users/tiago/OneDrive/Documents/GitHub/PlaceMark-2.0/node_modules/tsx/dist/esm/index.mjs:1:985)
  at Hooks.addCustomLoader (node:internal/modules/esm/hooks:183:24)
  at Hooks.register (node:internal/modules/esm/hooks:157:16)
  at async initializeHooks (node:internal/modules/esm/utils:318:5)
  at async customizedModuleWorker (node:internal/modules/esm/worker:109:13)
Node.js v22.14.0
```

[assistant] The error is clear – `--loader` is deprecated in Node v22. Use `--import` instead. Update your `package.json`:

```
```json
"test": "mocha --import tsx test/**/*.ts"
```
```

And update `.mocharc.cjs`:

```
```js
module.exports = {
 require: "tsx",
 spec: "test/**/*.ts",
 timeout: 10000
};
```
```

```
[user] $ npm run test
> placemark-2.0@0.0.1 test
> node --env-file=.env --import tsx/esm node_modules/mocha/bin/mocha.js --ui tdd test/**/*.ts
User Model tests
MONGO_URL mongodb+srv://tvillaca:20114785@placemarkcluster0.i0fobsj.mongodb.net/?appName=placemarkCluster0
connected to mongodb+srv://tvillaca:20114785@placemarkcluster0.i0fobsj.mongodb.net/?appName=placemarkCluster0
✓ create a user (57ms)
1 passing (1s)
```

[assistant] Your first test is passing! The connection to MongoDB Atlas is working and the user was created successfully.

Now you can add the remaining tests to `user-model-test.ts` one by one and keep running `npm run test` to verify each one passes.

[user] when I add your file, it goes an error

[assistant] Share the error and I'll fix it.

```
[user] node:internal/modules/run_main:122      triggerUncaughtException(      ^ Error: tsx must be loaded with --import instead of --loader The --
loader flag was deprecated in Node v20.6.0 and v18.19.0      at initialize (file:///C:/Users/tiago/OneDrive/Documents/GitHub/PlaceMark-2.0/node_
modules/tsx/dist/esm/index.mjs:1:985)      at Hooks.addCustomLoader (node:internal/modules/esm/hooks:183:24)      at Hooks.register (node:internal/
modules/esm/hooks:157:16)      at async initializeHooks (node:internal/modules/esm/utils:318:5)      at async customizedModuleWorker (node:internal/
modules/esm/worker:109:13) Node.js v22.14.0
```

[assistant] Your `.mocharc.cjs` is still using `loader`. Update it to:

```
```js
module.exports = {
 require: ["tsx/esm"],
 spec: "test/**/*.ts",
 timeout: 10000
};
```
```

[user] ✗ create a user

```
MongoParseError: Invalid scheme, expected connection string to start with "mongodb://" or "mongodb+srv://"
  at new ConnectionString (node_modules\mongodb-connection-string-url\src\index.ts:131:13)
  at parseOptions (node_modules\mongodb\src\connection_string.ts:256:15)
  at new MongoClient (node_modules\mongodb\src\mongo_client.ts:440:32)
  at NativeConnection.createClient (node_modules\mongoose\lib\drivers\node-mongodb-native\connection.js:338:14)
  at NativeConnection.openUri (node_modules\mongoose\lib\connection.js:1086:34)
```

```

    at Mongoose.connect (node_modules\mongoose\lib\mongoose.js:475:15)
    at dbConnect (src\lib\models\db.ts:31:18)
    at process.processTicksAndRejections (node:internal/process/task_queues:105:5)
    at async Context.<anonymous> (test\model\user-model-test.ts:11:5)
[assistant] The test can't read your `.env` file – `MONGO_URL` is coming through as `undefined` or empty. Check your test script in `package.json` – it needs `--env-file=.env`:

```

```

```json
"test": "node --env-file=.env --import tsx/esm node_modules/mocha/bin/mocha.js --ui tdd test/**/*.ts"
```

```

That's what was working before when your single test passed. Make sure that's still in your `package.json`.

[user] ✗ get a user - failures

CastError: Cast to ObjectId failed for value "123" (type string) at path "_id" for model "User"

```

    at SchemaObjectId.cast (node_modules\mongoose\lib\schema\objectId.js:252:11)
    at SchemaType.applySetters (node_modules\mongoose\lib\schemaType.js:1372:12)
    at SchemaType.castForQuery (node_modules\mongoose\lib\schemaType.js:1798:17)
    at cast (node_modules\mongoose\lib\cast.js:386:32)
    at Query.cast (node_modules\mongoose\lib\query.js:5212:12)
    at Query._castConditions (node_modules\mongoose\lib\query.js:2409:10)
    at model.Query._findOne (node_modules\mongoose\lib\query.js:2756:8)
    at model.Query.exec (node_modules\mongoose\lib\query.js:4797:80)
    at process.processTicksAndRejections (node:internal/process/task_queues:105:5)
    at async Object.findById (src\lib\models\mongo\user-store.ts:12:20) ✓ get a user - bad params
[assistant] The test is passing `"123"` as an ID, but Mongoose tries to cast it to an ObjectId and fails because `"123"` is not a valid ObjectId format.

```

You need to handle this in your `findOne` method in `user-store.ts`. Add a try/catch:

```

```typescript
async findOne(id: string): Promise<User | null> {
 if (!id) return null;
 try {
 return await UserMongoose.findById(id).lean();
 } catch (error) {
 return null;
 }
}
```

```

The `try/catch` catches the `CastError` and returns `null` instead of throwing, which is what your test expects.

[user] there is an error asking for other details, but those are optional

[assistant] Make those fields optional in your `User` interface in `placemark-types.ts`:

```

```typescript
export interface User {
 firstName: string;
 lastName: string;
 email: string;
 password: string;
 _id?: string;
 picture?: string;
 createdAt?: Date;
 createdBy?: string;
 superAdmin?: boolean;
}
```

```

...

The `?` makes them optional so TypeScript won't complain when they're not provided.

[user] get back to git flow. I will finish my feature

[assistant] Sure! When you're ready, the steps are:

```
```bash
git add .
git commit -m "feat: add user model and tests"
git flow feature finish mongodb
git push origin develop
```
```

[user] git push origin develop , explain thgis to me

[assistant] `git push origin develop` breaks down as:

- `git` – the git command
- `push` – send your local commits up to the remote repository
- `origin` – the name of your remote repository (GitHub in your case)
- `develop` – the branch you want to push

So in plain English: "send my local `develop` branch commits up to GitHub".

[assistant] Everything looks great! The push was successful. Here's what you can see in the graph now:

- `develop` and `origin` are in sync with "feat: add user model and tests"
- `dashboard` and `about view` is the previous commit below it
- `origin/main` is still on "First comit and starter pages" – that's fine, main only gets updated when you do a release in git flow

Your `develop` branch is now properly on GitHub.

[user] the graph has not changed

[assistant] That's expected – the graph looks the same because you committed directly to `develop` again (no feature branch was used).

To get the branching lines in the graph you need to follow the full git flow cycle:

```
```bash
git flow feature start my-feature # creates the branch
make changes
git add .
git commit -m "feat: something" # commit ON the feature branch
git flow feature finish my-feature # THEN merge back
```
```

You're only going to see that forked line in the graph when you start doing that correctly from the beginning. From your next feature onwards, stick to that cycle and the graph will start looking like the diagram you showed earlier.

[user] tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (feature/mongodb)

\$ git add .

warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tsconfig.json', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'test/setup.ts', LF will be replaced by CRLF the next time Git touches it

tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (feature/mongodb)

\$ git commit -m "feat: add user model and tests"

[feature/mongodb 0575fe0] feat: add user model and tests

13 files changed, 2511 insertions(+), 96 deletions(-)

create mode 100644 .mocharc.cjs

create mode 100644 src/hooks.server.ts

create mode 100644 src/lib/models/db.ts

```

create mode 100644 src/lib/models/mongo/user-store.ts
create mode 100644 src/lib/models/mongo/user.ts
create mode 100644 src/lib/types/placemark-types.ts
create mode 100644 test/fixtures.ts
create mode 100644 test/model/user-model-test.ts
create mode 100644 test/setup.ts
create mode 100644 test/test-utils.ts
tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (feature/mongodb)
$ git flow feature finish mongodb
Fetching from remote 'origin'...
Note: Could not fetch base branch 'develop': failed to fetch branch 'develop' from 'origin': fatal: couldn't find remote ref develop
Note: Could not fetch topic branch 'feature/mongodb': failed to fetch branch 'feature/mongodb' from 'origin': fatal: couldn't find remote ref feature/
mongodb
Fetch completed
Switched to branch 'develop'
Merging using strategy: merge
Successfully finished branch 'feature/mongodb' and updated 0 child base branches
tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (develop)
$ git push origin develop
Enumerating objects: 51, done.
Counting objects: 100% (51/51), done.
Delta compression using up to 14 threads
Compressing objects: 100% (32/32), done.
Writing objects: 100% (36/36), 22.34 KiB | 1.49 MiB/s, done.
Total 36 (delta 12), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (12/12), completed with 10 local objects.
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/villaca13/PlaceMark-2.0/pull/new/develop
remote:
To https://github.com/villaca13/PlaceMark-2.0.git
 * [new branch]      develop -> develop
tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (develop)
$ I did what you said
[assistant] Good progress! But I can see the same issue as before – the fetch warnings mean `feature/mongodb` was never pushed to `origin` before you
finished it, so the merge happened only locally with no branch line in the graph.

```

The fix going forward is to push your feature branch to origin **before** finishing it:

```

```bash
git flow feature start my-feature
make changes and commit
git push origin feature/my-feature # ← add this step
git flow feature finish my-feature
git push origin develop
```

```

Pushing the feature branch first ensures it exists on GitHub, which fixes the fetch warnings and will eventually also allow you to open **Pull Requests** on GitHub for code review before merging – which is standard practice on team projects.

```

[user] Ok , save it for later
[assistant] Got it, noted. What's next?
[user] how to use message channel in TS
[assistant] Can you give me a bit more context? Do you mean:

```

- **BroadcastChannel** – for communication between browser tabs?
- **MessageChannel** – for communication between two scripts/workers?

- ****Worker MessageChannel**** – for communication with Web Workers?
- ****Something else**** – like a messaging system between server and client in SvelteKit?
[user] singup form to communicate with +page.server.ts
[assistant] That's not MessageChannel – in SvelteKit that's done with ****Actions****. Here's how it works:

```
**`+page.svelte`** (the form):
```svelte
<!--form method="POST" action="?/signup">
 <input name="firstName" type="text" />
 <input name="lastName" type="text" />
 <input name="email" type="email" />
 <input name="password" type="password" />
 <button type="submit">Sign Up</button>
</form>
```

**`+page.server.ts`** (handles the form):
```typescript
import type { Actions } from "@sveltejs/kit";
import { userStore } from "$lib/models/mongo/user-store.js";

export const actions: Actions = {
 signup: async ({ request }) => {
 const form = await request.formData();
 const user = {
 firstName: form.get("firstName") as string,
 lastName: form.get("lastName") as string,
 email: form.get("email") as string,
 password: form.get("password") as string,
 };
 await userStore.add(user);
 return { success: true };
 },
};
```
```

The form `action="?/signup"` tells SvelteKit to send the form data to the `signup` action in `+page.server.ts` when submitted.

```
[user] ```typescript
import { placemarkService } from "$lib/services/placemark-service";
import { redirect, fail } from "@sveltejs/kit";
import { dev } from "$app/environment";

export const actions = {
  signup: async ({ request, cookies }) => {

    const form = await request.formData();
    const firstName = form.get("firstName") as string;
    const lastName = form.get("lastName") as string;
    const email = form.get("email") as string;
    const password = form.get("password") as string;

    if (email === "" || password === "" || firstName === "" || lastName === "") {
      throw redirect(307, "/signup");
    } else {
      console.log(`attempting to sign up email: ${email} with password: ${password}`);
      const session = await placemarkService.signup({ firstName, lastName, email, password });
    }
  }
};
```
```



```

 if (session) {
 const userJson = JSON.stringify(session);
 cookies.set("placemark-user", userJson, {
 path: "/",
 httpOnly: true,
 sameSite: "strict",
 secure: !dev,
 maxAge: 60 * 60 * 24 * 7 // one week
 });
 throw redirect(303, "/signup");
 } else {
 throw redirect(307, "/signup");
 }
 }
}
}
}

```

[assistant] This looks good overall. A few things to note:

**\*\*1. Missing types\*\*** – add the `Actions` type:

```

```typescript
import type { Actions } from "@sveltejs/kit";

export const actions: Actions = {
  ...

```

****2. Redirect after successful signup**** – you're redirecting back to `/signup` on success, which is likely wrong:

```

```typescript
throw redirect(303, "/dashboard"); // should go to dashboard after signup
...

```

**\*\*3. `throw redirect` is deprecated in newer SvelteKit\*\*** – use `redirect` directly:

```

```typescript
import { redirect, fail } from "@sveltejs/kit";

// instead of throw redirect(...)
redirect(303, "/dashboard");
...

```

****4. Empty field check**** – consider using `fail` instead of redirect when validation fails, so you can show an error message to the user:

```

```typescript
if (email === "" || password === "" || firstName === "" || lastName === "") {
 return fail(400, { message: "All fields are required" });
}
...

```

What does your `placemarkService.signup` do? That will help me check if the session handling is correct.

```

[user] ```typescript
async signup(user: User): Promise<boolean> {
 try {
 user.superAdmin = true;
 const newUser = await userStore.add(user);
 console.log("user created", newUser.email);
 return !!newUser;
 } catch (error) {
 console.log(error);
 }
}

```

```

 return false;
 }
},
...

```

explain this function to me?

[assistant] Sure!

`**`async signup(user: User): Promise<boolean>`**` – takes a ``User`` object and returns a Promise that resolves to a boolean (true or false).

`**`user.superAdmin = true`**` – sets every signed up user as a superAdmin. This is probably temporary for development – in a real app you wouldn't want every user to be an admin.

`**`const newUser = await userStore.add(user)`**` – adds the user to MongoDB and waits for the result.

`**`console.log("user created", newUser.email)`**` – logs the new user's email to confirm it was created.

`**`return !!newUser`**` – converts ``newUser`` to a boolean. If ``newUser`` exists it returns ``true``, if it's ``null`` or ``undefined`` it returns ``false``. The ``!!`` is a common JS/TS trick to convert any value to a boolean.

`**`catch (error)`**` – if anything goes wrong (e.g. database error, duplicate email), it catches the error, logs it, and returns ``false`` instead of crashing.

So in plain English: "Try to create a user in the database. If it works return true, if anything goes wrong return false."

One thing to note – your ``+page.server.ts`` is storing the session in a cookie with ``JSON.stringify(session)``, but ``signup`` only returns a boolean, not a user object. So ``userJson`` would just be ``"true"`` which isn't very useful. You may want to return the ``newUser`` object instead of ``!!newUser``.

[user] how to return an error if user already exists

[assistant] Update your ``signup`` function in ``placemark-service.ts`` to check first:

```

```typescript
async signup(user: User): Promise<User | null>; {
  try {
    const existingUser = await userStore.findByEmail(user.email);
    if (existingUser) {
      console.log("user already exists");
      return null;
    }
    user.superAdmin = true;
    const newUser = await userStore.add(user);
    console.log("user created", newUser.email);
    return newUser;
  } catch (error) {
    console.log(error);
    return null;
  }
},
...

```

Then in ``+page.server.ts`` handle the null response:

```

```typescript
const newUser = await placemarkService.signup({ firstName, lastName, email, password });

if (newUser) {
 const userJson = JSON.stringify(newUser);

```

```

 cookies.set("placemark-user", userJson, { ... });
 redirect(303, "/dashboard");
 } else {
 return fail(400, { message: "Email already exists" });
 }
}

```

And in your `+page.svelte` you can display the error:

```

```svelte
<script lang="ts">
  const { form } = $props();
</script>

{#if form?.message}
  <p class="has-text-danger">{form.message}</p>
{/if}

[user] ```typescript
import {fail} from "@sveltejs/kit";

```

how to used ?

[assistant] `fail` returns an error response from a form action with a status code and data. Here's how it works:

```

```typescript
// in +page.server.ts
return fail(400, { message: "Email already exists" });

```

The two arguments are:

- `400` – the HTTP status code (400 = bad request)
- `{ message: "..."}`  – any data you want to send back to the form

Then in your `+page.svelte` you access it via the `form` prop:

```

```svelte
<script lang="ts">
  const { form } = $props();
</script>

{#if form?.message}
  <p class="has-text-danger">{form.message}</p>
{/if}

<form method="POST" action="/signup">
  ...
</form>

```

So the flow is:

1. User submits form
2. Action runs in `+page.server.ts`
3. If something is wrong, `return fail(400, { message: "..."} )`
4. SvelteKit sends that data back to the page
5. `form.message` displays the error to the user

```
[user] ```svelte
  import Message from "$lib/ui/Message.svelte";
  ```
```

are only red messages ?

[assistant] No, it depends on how you build your `Message.svelte` component. You can pass a type or color as a prop to show different styles. For example using Bulma:

```
```svelte
<!-- Message.svelte -->
<script lang="ts">
  const { message, type = "is-danger" } = $props();
</script>

{#if message}
  <div class="notification {type}">
    {message}
  </div>
{/if}
```
```

Then use it with different types:

```
```svelte
<!-- red -->
<Message message="Email already exists" type="is-danger" />

<!-- green -->
<Message message="User created!" type="is-success" />

<!-- yellow -->
<Message message="Please fill all fields" type="is-warning" />

<!-- blue -->
<Message message="Welcome back!" type="is-info" />
```
```

These are Bulma's built-in notification colours. Share your current `Message.svelte` and I can tell you exactly what it supports.

[user] how do I know which one of the database I am accessing when using this

```
```dotenv
MONGO_URL=mongodb+srv://tvillaca:20114785@placemarkcluster0.i0fobsj.mongodb.net/?appName=placemarkCluster0
```

[assistant] Your connection string doesn't specify a database name, so Mongoose uses `test` by default – which matches what you see in Atlas (`test` with `routes`, `stops`, `users`).

To explicitly choose a database, add the name to your URL:

```
```dotenv
MONGO_URL=mongodb+srv://tvillaca:20114785@placemarkcluster0.i0fobsj.mongodb.net/placemark?appName=placemarkCluster0
```

The database name goes between the `/` and `?` – in this case `placemark`. That will create and use a `placemark` database instead of `test`.

[user] I finished this feature, let do git flow

```
[assistant] ```bash
git add .
git commit -m "feat: add signup with duplicate email check"
git flow feature finish signup
git push origin develop
```
```

[user] there are more steps. I want to see it on the graph

[assistant] You're right! To see it on the graph you need to push the feature branch to origin **before** finishing it:

```
```bash
git add .
git commit -m "feat: add signup with duplicate email check"
git push origin feature/signup
git flow feature finish signup
git push origin develop
```
```

The `git push origin feature/signup` step is what creates the branch line in the graph.

[user] tiago@PavVillaca MINGW64 ~/OneDrive/Documents/GitHub/PlaceMark-2.0 (feature/logging_in_auth)

```
$ git push origin feature/signup
```

```
error: src refspec feature/signup does not match any
```

```
error: failed to push some refs to 'https://github.com/villaca13/PlaceMark-2.0.git'
```

[assistant] The error is because your current branch is `feature/logging_in_auth`, not `feature/signup`. Push the correct branch:

```
```bash
git push origin feature/logging_in_auth
```
```

[user] made no difference on the graph

[assistant] Still a straight line because the feature branch had commits but was merged with a **fast-forward** strategy, which doesn't create a visible branch line.

To force a merge commit (which creates the fork in the graph), you need to tell git flow to use `--no-ff`. You can configure this in your `.gitconfig` or `git flow` config:

```
```bash
git config gitflow.feature.finish.no-ff true
```
```

Run that once and from now on every `git flow feature finish` will create a proper merge commit, giving you the branching line in the graph.

[user] i have finish the feature mongodb_stops

[assistant] Follow these steps in order:

```
```bash
git add .
git commit -m "feat: add stops model and tests"
git push origin feature/mongodb_stops
git flow feature finish mongodb_stops
git push origin develop
```
```

Remember we set `--no-ff` so this time you should see the branch line in the graph!

[user] cool. getting better. how does it work for main. I wanted to see main, develop and features separated

[assistant] You can see the pink branch line now – that's the feature branch merging in.

For `main` to show separately, in git flow you only merge into `main` when you do a **release**. The flow is:

```
```bash
```

```
git flow release start 1.0.0 # branches off develop
do any final fixes
git flow release finish 1.0.0 # merges into BOTH main and develop, creates a tag
git push origin main
git push origin develop
git push origin --tags
```

That will give you a graph that looks like:

```

...
main ██████████ (v1.0.0 tag)
 /
develop ██████████ \pink/ \pink/
 ● ●
features
...

```

So the full picture is:

- ```
- **Features** branch off and merge back into `develop`
- **Releases** branch off `develop` and merge into both `develop` and `main`
- **Hotfixes** branch off `main` for emergency fixes
```

You're doing the features correctly now. When you feel `develop` is stable enough to ship, do a release and `main` will update.

```
[user] declare an array on interfaces
[assistant] ```typescript
export interface User {
  firstName: string;
  stops: string[];           // array of strings
  coords: number[];         // array of numbers
  tags: string[];           // array of strings
}
```

Or if the array holds objects:

```

``typescript
export interface Route {
  title: string;
  stops: Stop[];           // array of Stop objects
}

```

```
[user] ```typescript
  stops[]: {Types.ObjectId | User} ,
  ```
```

[assistant] That syntax is wrong. Here's the correct way:

```
``typescript
stops: Types.ObjectId[];
...

```

Or if it can be either an ObjectId or a populated User:

```
``typescript
stops: (Types.ObjectId | User)[];
```

```
[user] ```typescript
stops: [{
 type: Schema.Types.ObjectId,
 ref: "Stop",
},],
```
```

for the routeSchema

[assistant] That's almost correct but the array syntax for Mongoose schemas should be:

```
```typescript
stops: [{ type: Schema.Types.ObjectId, ref: "Stop" }]
```
```

No trailing comma inside the array. This tells Mongoose that `stops` is an array of ObjectId references to the `Stop` model.

```
[user] ```typescript
async deleteStopFromRoute(routeId, stopId): Promise<Route | null>; {
  const routeToBeUpdated = await RouteMongoose.findOne({ _id: routeId });
  routeToBeUpdated.stops.pull(stopId);
  const routeReponse = await routeToBeUpdated.save();
  return this.getRouteById(routeReponse._id);
},
```
```

[assistant] Here's the TypeScript version with proper types:

```
```typescript
async deleteStopFromRoute(routeId: string, stopId: string): Promise<Route | null>; {
  try {
    const routeToBeUpdated = await RouteMongoose.findOne({ _id: routeId });
    if (!routeToBeUpdated) return null;
    routeToBeUpdated.stops.pull(stopId);
    const routeResponse = await routeToBeUpdated.save();
    return this.getRouteById(routeResponse._id);
  } catch (error) {
    console.log(error);
    return null;
  }
},
```
```

Changes:

- Added `string` types to both parameters
- Added null check for `routeToBeUpdated` – if the route doesn't exist, `.stops.pull()` would crash
- Wrapped in `try/catch` for error handling
- Fixed typo `routeReponse` → `routeResponse`

[assistant] The error is because TypeScript doesn't know that `.pull()` exists on your `stops` array type. It's a Mongoose method, not a standard array method.

Fix your `Route` interface to use `Types.DocumentArray`:

```
```typescript
import { Types } from "mongoose";

export interface Route {
  ...
  stops: Types.ObjectId[];
}
```

```
}  
```
```

And in your schema, make sure the type is correct. Then cast it in the function:

```
```typescript  
const routeToBeUpdated = await RouteMongoose.findOne({ _id: routeId });  
if (!routeToBeUpdated) return null;  
(routeToBeUpdated.stops as any).pull(stopId);  
```
```

The `as any` is a quick workaround since `.pull()` is a Mongoose-specific method that TypeScript doesn't know about on a regular array type.  
[assistant] ESLint is complaining about `as any`. Use a more specific type instead:

```
```typescript  
(routeToBeUpdated.stops as Types.DocumentArray<Types.ObjectId>).pull(stopId);  
```
```

Make sure `Types` is imported from mongoose:

```
```typescript  
import { Types } from "mongoose";  
```
```

[user] i finished my feature mongodb\_routes  
[assistant] Follow the steps:

```
```bash  
git add .  
git commit -m "feat: add routes model and tests"  
git push origin feature/mongodb_routes  
git flow feature finish mongodb_routes  
git push origin develop  
```
```

[user] let do a release now  
[assistant] ````bash`  
git flow release start 1.0.0  
git flow release finish 1.0.0  
git push origin main  
git push origin develop  
git push origin --tags  
`````

When you run `git flow release finish 1.0.0` it will open a text editor asking for a tag message – just type something like `Release 1.0.0` and save it.

[user] now I want to switch to develop

[assistant] ````bash`
git checkout develop
`````

[user] how to show one line for main and one line for develop?

[assistant] Your graph is actually correct and looking great now! You can see the coloured branch lines (pink, green) for the features merging in.

``main`` and ``develop`` are on the same line at the top because they're currently on the same commit after the release. They will **separate** when you start adding new commits to ``develop`` – at that point ``develop`` will move ahead while ``main`` stays at ``1.0.0``.

So after your next feature is merged into ``develop``, the graph will show them on separate lines.

[user] I meant the vertical line



[assistant] You can't have two separate vertical lines for `main` and `develop` in this view because they're pointing to the same commit right now.

Once you start a new feature and merge it into `develop`, the graph will automatically split into two vertical lines – one for `main` staying at `1.0.0` and one for `develop` moving ahead.

[user] i have finished my feature stops\_page , git flow

```
[[[assistant] ```bash
git add .
git commit -m "feat: add stops page"
git push origin feature/stops_page
git flow feature finish stops_page
git push origin develop
```
```

[user] i have finished my feature dashboard_page

```
[[[assistant] ```bash
git add .
git commit -m "feat: add dashboard page"
git push origin feature/dashboard_page
git flow feature finish dashboard_page
git push origin develop
```
```

[user] i finished my feature route\_page

```
[[[assistant] ```bash
git add .
git commit -m "feat: add route page"
git push origin feature/route_page
git flow feature finish route_page
git push origin develop
```
```

[user] finish my feature cleaning_up

```
[[[assistant] ```bash
git add .
git commit -m "feat: cleaning up"
git push origin feature/cleaning_up
git flow feature finish cleaning_up
git push origin develop
```
```

[user] finished my feature dropdown\_menu

```
[[[assistant] ```bash
git add .
git commit -m "feat: add dropdown menu"
git push origin feature/dropdown_menu
git flow feature finish dropdown_menu
git push origin develop
```
```

[user] how to do a hot fix

[assistant] A hotfix is for urgent fixes directly on `main`. Here's how:

```
[[[assistant] ```bash
git flow hotfix start 1.0.1
# make your fix
git add .
git commit -m "fix: your fix description"
git flow hotfix finish 1.0.1
git push origin main
git push origin develop
git push origin --tags
```
```

...

It branches off `main`, and when finished it merges into **both** `main` and `develop` automatically – so your fix is in both places.

[user] lets do a release first then

[assistant] ```bash

git flow release start 1.1.0

git flow release finish 1.1.0

git push origin main

git push origin develop

git push origin --tags

```

[user] hot fix now

[assistant] ```bash

git flow hotfix start 1.1.1

make your fix

git add .

git commit -m "fix: your fix description"

git flow hotfix finish 1.1.1

git push origin main

git push origin develop

git push origin --tags

```

[user] finished my hotfix message type

[assistant] ```bash

git add .

git commit -m "fix: message type"

git flow hotfix finish message\_type

git push origin main

git push origin develop

git push origin --tags

```

[user] do I need to merge main into develop ?

[assistant] No, git flow does that **automatically** when you run `git flow hotfix finish`. It merges the hotfix into both `main` and `develop` for you – that's exactly what you see in the graph: "Merge branch 'main' into develop".

So everything is correct, you don't need to do anything extra.

[user] why they are no in the same line ?

[assistant] Because `main` and `develop` now have **different commits** on them:

- **main** is at the hotfix commit (`fix: message type`, tag `1.1.1`)

- **develop** is one commit ahead with the "Merge branch 'main' into develop" commit

They diverged after the hotfix, which is why they show on separate lines. This is actually **correct git flow behaviour** – they'll come back together on the same line when you do your next release, which merges `develop` back into `main`.

[user] but they are equal >

[assistant] You're right – they are equal in terms of code, but git sees them as different commits because the merge created a new commit on `develop`.

It will resolve itself visually once you do your next release. Don't worry about it.

[user] switch to develop now then

[assistant] ```bash

git checkout develop

```

[user] I finish my feature users\_details

[assistant] ```bash

git add .

```
git commit -m "feat: add user details"
git push origin feature/users_details
git flow feature finish users_details
git push origin develop
```
```

[user] Hashing & salting passwords
[assistant] This is done using **bcrypt**. Install it:

```
```bash
npm install bcrypt
npm install --save-dev @types/bcrypt
```
```

Then in your `user-store.ts` or `placemark-service.ts`, hash the password before saving:

```
```typescript
import bcrypt from "bcrypt";

// when creating a user
async add(user: User): Promise<User> {
 user.password = await bcrypt.hash(user.password, 10);
 const newUser = new UserMongoose(user);
 return newUser.save();
}
```
```

And when logging in, compare the password:

```
```typescript
// when logging in
async login(email: string, password: string): Promise<User | null> {
 const user = await UserMongoose.findOne({ email }).lean();
 if (!user) return null;
 const isValid = await bcrypt.compare(password, user.password);
 if (!isValid) return null;
 return user;
}
```
```

The `10` in `bcrypt.hash` is the **salt rounds** – the higher the number the more secure but slower. 10 is the standard for most apps.

[user] OAuth (login via Github, Google etc)

[assistant] In SvelteKit the easiest way is using **Auth.js** (formerly NextAuth). Install it:

```
```bash
npm install @auth/sveltekit
```
```

Then create `src/auth.ts`:

```
```typescript
import { SvelteKitAuth } from "@auth/sveltekit";
import GitHub from "@auth/sveltekit/providers/github";
import Google from "@auth/sveltekit/providers/google";

export const { handle, signIn, signOut } = SvelteKitAuth({
 providers: [
```

```

 GitHub({
 clientId: process.env.GITHUB_ID,
 clientSecret: process.env.GITHUB_SECRET,
 }),
 Google({
 clientId: process.env.GOOGLE_ID,
 clientSecret: process.env.GOOGLE_SECRET,
 }),
],
});

```

Add to your ``.env``:

```

```dotenv
GITHUB_ID=your_github_client_id
GITHUB_SECRET=your_github_client_secret
GOOGLE_ID=your_google_client_id
GOOGLE_SECRET=your_google_client_secret
AUTH_SECRET=any_random_string
```

```

In your ``.hooks.server.ts``:

```

```typescript
export { handle } from "./auth";
```

```

Then in your login page:

```

```svelte
<script lang="ts">
  import { signIn, signOut } from "@auth/sveltekit/client";
</script>

<button on:click={() => signIn("github")}>Login with GitHub</button>
<button on:click={() => signIn("google")}>Login with Google</button>
```

```

To get the credentials from GitHub/Google you need to create OAuth apps on their developer portals – **GitHub**: Settings → Developer Settings → OAuth Apps, **Google**: Google Cloud Console → Credentials.  
 [user] 3rd Party Auth Service (Auth0, Firebase or equivalent)  
 [assistant] The most popular options are:

#### **Auth0**

- Most feature-rich, supports social logins, MFA, user management dashboard
- Free tier: 7,500 active users
- Install: ``npm install @auth0/auth0-spa-js``

#### **Firebase Auth**

- Easy to set up, integrates well with other Google services
- Free tier: unlimited auth users
- Install: ``npm install firebase``

#### **Clerk**

- Easiest to set up, best UI components out of the box
- Free tier: 10,000 monthly active users
- Install: ``npm install @clerk/clerk-js``

#### **Supabase Auth**

- Open source alternative, supports social logins
- Free tier: 50,000 monthly active users
- Install: ``npm install @supabase/supabase-js``

For a SvelteKit project I'd recommend **Clerk** if you want the quickest setup, or **Auth0** if you need the most control and features.

Which one are you interested in?

[user] Clerk

[assistant] Install it:

```
```bash
npm install @clerk/clerk-js
```
```

Add to your ``.env``:

```
```dotenv
PUBLIC_CLERK_PUBLISHABLE_KEY=your_publishable_key
CLERK_SECRET_KEY=your_secret_key
```
```

Get those keys from [clerk.com](https://clerk.com) → Create application → API Keys.

Then in your ``src/hooks.server.ts``:

```
```typescript
import { CLERK_SECRET_KEY } from "$env/static/private";
import { createClerkClient } from "@clerk/clerk-js";

const clerk = createClerkClient({ secretKey: CLERK_SECRET_KEY });

export const handle = async ({ event, resolve }) => {
  const session = await clerk.sessions.getSession(
    event.cookies.get("__session") ?? ""
  );
  event.locals.user = session?.user ?? null;
  return resolve(event);
};
```
```

In your ``+page.svelte``:

```
```svelte
<script lang="ts">
  import { PUBLIC_CLERK_PUBLISHABLE_KEY } from "$env/static/public";
  import Clerk from "@clerk/clerk-js";

  const clerk = new Clerk(PUBLIC_CLERK_PUBLISHABLE_KEY);

  async function load() {
    await clerk.load();
  }

  load();
</script>

<button on:click={() => clerk.openSignIn()}>Sign In</button>
<button on:click={() => clerk.openSignUp()}>Sign Up</button>
<button on:click={() => clerk.signOut()}>Sign Out</button>
```
```

...

Clerk handles everything – UI, sessions, social logins – without you needing to build any of it yourself.

[user] Auth0 ? they are all free ?

[assistant] Yes, all have free tiers but with limits. Auth0's free tier gives you **7,500 active users** and **2 social connections** (e.g. GitHub + Google) which is plenty for a project.

Install it:

```
```bash
npm install @auth0/auth0-spa-js
```
```

Get your credentials from [auth0.com](https://auth0.com) → Create Application → Single Page Application → Settings:

Add to your `.env`:

```
```dotenv
PUBLIC_AUTH0_DOMAIN=your_domain.auth0.com
PUBLIC_AUTH0_CLIENT_ID=your_client_id
AUTH0_CLIENT_SECRET=your_client_secret
```
```

In your `+page.svelte`:

```
```svelte
<script lang="ts">
  import { PUBLIC_AUTH0_DOMAIN, PUBLIC_AUTH0_CLIENT_ID } from "$env/static/public";
  import { createAuth0Client } from "@auth0/auth0-spa-js";

  let auth0;

  async function load() {
    auth0 = await createAuth0Client({
      domain: PUBLIC_AUTH0_DOMAIN,
      clientId: PUBLIC_AUTH0_CLIENT_ID,
    });
  }

  load();
</script>

<button on:click={() => auth0.loginWithPopup()}>Login</button>
<button on:click={() => auth0.logout()}>Logout</button>
```
```

Compared to Clerk:

- **Auth0** – more control, more config, better for enterprise
- **Clerk** – easier setup, better UI out of the box, less config

For a college project either works fine. Which are you going with?

[user] Image Gallery Feature , clodnary ?

[assistant] Yes, Cloudinary is the most popular choice for image management. Free tier gives you **25GB storage** and **25GB bandwidth** per month.

Install it:

```
```bash
npm install cloudinary
```
```

...

Add to your ``.env``:

```
```.env
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```.env
```

Get those from [cloudinary.com](https://cloudinary.com) → Dashboard.

Create ``src/lib/services/cloudinary-service.ts``:

```
```.typescript
import { v2 as cloudinary } from "cloudinary";
import { CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET } from "$env/static/private";

cloudinary.config({
  cloud_name: CLOUDINARY_CLOUD_NAME,
  api_key: CLOUDINARY_API_KEY,
  api_secret: CLOUDINARY_API_SECRET,
});

export const imageStore = {
  async uploadImage(imagePath: string): Promise<string> {
    const result = await cloudinary.uploader.upload(imagePath);
    return result.url;
  },

  async deleteImage(imageId: string): Promise<void> {
    await cloudinary.uploader.destroy(imageId);
  },

  async getAllImages(): Promise<string[]> {
    const result = await cloudinary.api.resources();
    return result.resources.map((r: any) => r.url);
  },
};
```

In your ``+page.server.ts`` to handle image upload:

```
```.typescript
import { imageStore } from "$lib/services/cloudinary-service";

export const actions = {
 upload: async ({ request }) => {
 const form = await request.formData();
 const file = form.get("image") as File;
 const buffer = await file.arrayBuffer();
 const base64 = Buffer.from(buffer).toString("base64");
 const dataUri = `data:${file.type};base64,${base64}`;
 const url = await imageStore.uploadImage(dataUri);
 return { url };
 },
};
```

In your ``+page.svelte``:

```

```svelte
<form method="POST" action="?/upload" enctype="multipart/form-data">
  <input type="file" name="image" accept="image/*" />
  <button type="submit">Upload</button>
</form>

{#if form?.url}
  <img src={form.url} alt="uploaded" />
{/if}
```

```

Want me to show how to display a gallery of all uploaded images too?

[user] I have finished my feature sanitisation

[assistant] ```bash

```

git add .
git commit -m "feat: add sanitisation"
git push origin feature/sanitisation
git flow feature finish sanitisation
git push origin develop
```

```

[user] intelli sense is not working in VS code

[assistant] Try these fixes in order:

****1. Restart TS Server**** – most common fix:

`Ctrl+Shift+P` → type `Restart TS Server` → Enter

****2. Reload VS Code window:****

`Ctrl+Shift+P` → type `Reload Window` → Enter

****3. Check your `tsconfig.json` is valid** – a syntax error there kills IntelliSense.

****4. Make sure TypeScript is installed:****

```

```bash
npm install --save-dev typescript
```

```

****5. Check VS Code is using the workspace TypeScript**** – bottom right of VS Code you'll see a version number like `TS 5.x.x`. Click it and select ****"Use Workspace Version"**** instead of VS Code's built-in version.

Which of these did you try?

[user] finished password_hashing

[assistant] ```bash

```

git add .
git commit -m "feat: add password hashing"
git push origin feature/password_hashing
git flow feature finish password_hashing
git push origin develop
```

```

[user] ```typescript

```

import { dbConnect } from "$lib/models/db";
import type { Handle } from "@sveltejs/kit";

```

```

await dbConnect();
export const handle: Handle = async ({ event, resolve }) => {
 const response = await resolve(event);

```



```

 return response;
 });
}

```

how do I edit this for the Auth google and git ?

[assistant] You need to combine your existing `handle` with Auth.js's `handle` using SvelteKit's `sequence` helper:

```

```typescript
import { dbConnect } from "$lib/models/db";
import { SvelteKitAuth } from "@auth/sveltekit";
import GitHub from "@auth/sveltekit/providers/github";
import Google from "@auth/sveltekit/providers/google";
import { GITHUB_ID, GITHUB_SECRET, GOOGLE_ID, GOOGLE_SECRET } from "$env/static/private";
import type { Handle } from "@sveltejs/kit";
import { sequence } from "@sveltejs/kit/hooks";

await dbConnect();

const authHandle = SvelteKitAuth({
  providers: [
    GitHub({ clientId: GITHUB_ID, clientSecret: GITHUB_SECRET }),
    Google({ clientId: GOOGLE_ID, clientSecret: GOOGLE_SECRET }),
  ],
});

const customHandle: Handle = async ({ event, resolve }) => {
  const response = await resolve(event);
  return response;
};

export const handle = sequence(authHandle, customHandle);
```

```

`sequence` chains multiple handle functions together so they both run on every request.

[assistant] You need to add those variables to your `.env` file:

```

```dotenv
GITHUB_ID=your_github_client_id
GITHUB_SECRET=your_github_client_secret
GOOGLE_ID=your_google_client_id
GOOGLE_SECRET=your_google_client_secret
AUTH_SECRET=any_random_string
```

```

Get the values from:

- **GitHub**: Settings → Developer Settings → OAuth Apps → New OAuth App

- **Google**: [console.cloud.google.com](https://console.cloud.google.com) → Credentials → Create OAuth Client ID

[assistant] Those are just placeholder values – you need to replace them with your actual credentials from GitHub and Google.

**GitHub:**

1. Go to github.com → Settings → Developer Settings → OAuth Apps → New OAuth App
2. Set Homepage URL: `http://localhost:5173`
3. Set Callback URL: `http://localhost:5173/auth/callback/github`
4. Copy the **Client ID** and **Client Secret**

**Google:**

1. Go to console.cloud.google.com → Create Project
2. APIs & Services → Credentials → Create OAuth Client ID
3. Set Authorised redirect URI: `http://localhost:5173/auth/callback/google`
4. Copy the **Client ID** and **Client Secret**

**AUTH\_SECRET** – generate a random string:

```
```bash
openssl rand -base64 32
```
```

Then replace the placeholders in your `.env` with the real values.

[assistant] Svelte 5 deprecated `on:click`. Use `onclick` instead:

```
```svelte
<button onclick={() => signIn("google")}>Login with Google</button>
<button onclick={() => signIn("github")}>Login with GitHub</button>
```
```

[user] let implement now Auth0

[assistant] First, install it:

```
```bash
npm install @auth0/auth0-spa-js
```
```

Add to your `.env`:

```
```dotenv
PUBLIC_AUTH0_DOMAIN=your_domain.auth0.com
PUBLIC_AUTH0_CLIENT_ID=your_client_id
AUTH0_CLIENT_SECRET=your_client_secret
```
```

Get those from [auth0.com](https://auth0.com) → Create Application → Regular Web Application → Settings.

Set the following in Auth0 dashboard:

- **Allowed Callback URLs**: `http://localhost:5173/callback`
- **Allowed Logout URLs**: `http://localhost:5173`
- **Allowed Web Origins**: `http://localhost:5173`

Are you replacing the GitHub/Google auth you just set up, or adding Auth0 alongside it?

[user] Adding

[assistant] Then you can add Auth0 as another provider in your existing `hooks.server.ts` using Auth.js, which supports Auth0 natively:

```
```bash
npm install @auth/sveltekit
```
```

Add to your `.env`:

```
```dotenv
AUTH0_ID=your_client_id
AUTH0_SECRET=your_client_secret
AUTH0_ISSUER=https://your_domain.auth0.com
```
```

Then update `hooks.server.ts`:

```
```typescript
```

```
import Auth0 from "@auth/sveltekit/providers/auth0";

const authHandle = SvelteKitAuth({
  providers: [
    GitHub({ clientId: GITHUB_ID, clientSecret: GITHUB_SECRET }),
    Google({ clientId: GOOGLE_ID, clientSecret: GOOGLE_SECRET }),
    Auth0({
      clientId: AUTH0_ID,
      clientSecret: AUTH0_SECRET,
      issuer: AUTH0_ISSUER
    })
  ],
});
```

And add the imports:

```
```typescript
import { GITHUB_ID, GITHUB_SECRET, GOOGLE_ID, GOOGLE_SECRET, AUTH0_ID, AUTH0_SECRET, AUTH0_ISSUER } from "$env/static/private";
import Auth0 from "@auth/sveltekit/providers/auth0";
```
```

Then in your login page add the button:

```
```svelte
<button onclick={() => signIn("auth0")}>Login with Auth0</button>
```
```

[user] where to do this ? And add the imports:

[assistant] In your `hooks.server.ts` at the top of the file, alongside your existing imports:

```
```typescript
import { dbConnect } from "$lib/models/db";
import { SvelteKitAuth } from "@auth/sveltekit";
import GitHub from "@auth/sveltekit/providers/github";
import Google from "@auth/sveltekit/providers/google";
import Auth0 from "@auth/sveltekit/providers/auth0";
import { GITHUB_ID, GITHUB_SECRET, GOOGLE_ID, GOOGLE_SECRET, AUTH0_ID, AUTH0_SECRET, AUTH0_ISSUER } from "$env/static/private";
import type { Handle } from "@sveltejs/kit";
import { sequence } from "@sveltejs/kit/hooks";
```
```

All imports always go at the very top of the file before any other code.

[user] i finished my feature auth_google_git

[assistant] ```bash

```
git add .
git commit -m "feat: add Google and GitHub authentication"
git push origin feature/auth_google_git
git flow feature finish auth_google_git
git push origin develop
```
```

[user] lets do a release now

[assistant] ```bash

```
git flow release start 1.2.0
git flow release finish 1.2.0
git push origin main
git push origin develop
git push origin --tags
```
```

```
[user] 2.0.0 please
[assistant] ```bash
git flow release start 2.0.0
git flow release finish 2.0.0
git push origin main
git push origin develop
git push origin --tags
```
```

```
[user] move back to develop
[assistant] ```bash
git checkout develop
```
```

[user] Image Gallery Feature , do you know what would it mean as a feature for webapp ?
[assistant] Yes! For a webapp like PlaceMark, an Image Gallery feature would mean:

****For Stops**** – each stop could have multiple images attached to it, for example:

- A photo of the bus stop
- A photo of the surrounding area
- A photo of the stop sign/number

****For Routes**** – a route could have a cover image or a collection of photos along the route

****What it involves:****

- ****Upload**** – a form to upload images (single or multiple)
- ****Storage**** – saving images somewhere (Cloudinary in your case)
- ****Display**** – showing images in a gallery grid on the stop/route page
- ****Delete**** – removing images from the gallery and Cloudinary
- ****Associate**** – linking images to a specific stop or route in MongoDB

****In your data model**** it would mean adding an `images` array to your `Stop` or `Route` interface:

```
```typescript
export interface Stop {
 ...
 images: string[]; // array of Cloudinary URLs
}
```
```

Is this for stops, routes, or both?

[user] what could be the difference between these two items in my assignment: Multiple images and Image gallery feature

[assistant] Good question. They could mean:

****Multiple Images****

- Basic ability to upload and store more than one image per stop/route
- Just the data model and upload functionality
- `images: string[]` in your schema
- A simple upload form that accepts multiple files

****Image Gallery Feature****

- A polished UI experience for viewing those images
- Thumbnail grid layout
- Click to enlarge / lightbox effect
- Navigation between images (next/previous)
- Delete individual images
- Maybe drag to reorder images

So in short:

- **Multiple images** = the backend/data side – storing more than one image
- **Image gallery** = the frontend/UI side – displaying and managing those images in a visually appealing way

They build on each other – you need multiple images support first, then the gallery is the UI layer on top of it.

[user] To prevent the map being truncated, set its height: what means truncated in this context

[assistant] Truncated in this context means the map is **cut off** or **not fully visible** – it renders but only shows a partial view, like the bottom half is missing or it collapses to 0 height.

This happens because map libraries like Leaflet need an explicit height set on the container element, otherwise the browser doesn't know how tall to make it and it either collapses or gets clipped.

Fix it with:

```
```css
.map {
 height: 400px; /* or 100vh for full screen */
 width: 100%;
}
```
```

[user] i finished my feature maps

[assistant] ```bash

```
git add .
git commit -m "feat: add maps"
git push origin feature/maps
git flow feature finish maps
git push origin develop
```
```

[user] i finished my feature pictures

[assistant] ```bash

```
git add .
git commit -m "feat: add pictures"
git push origin feature/pictures
git flow feature finish pictures
git push origin develop
```
```

[user] for in TS

[assistant] ```typescript

```
for (let i = 0; i < array.length; i++) {
  console.log(array[i]);
}
```

Or the cleaner `for...of`:

```
```typescript
for (const item of array) {
 console.log(item);
}
```

Or `forEach`:

```
```typescript
array.forEach((item) => {
  console.log(item);
});
```

[user] open weather API
[assistant] Install the package:

```
```bash
npm install openweathermap-ts
```
```

Get your API key from openweathermap.org → Sign up → API Keys.

Add to your ``.env``:

```
```dotenv
OPENWEATHER_API_KEY=your_api_key
```
```

Create ``src/lib/services/weather-service.ts``:

```
```typescript
import { OPENWEATHER_API_KEY } from "$env/static/private";

export const weatherService = {
 async getWeather(lat: number, lon: number) {
 const response = await fetch(
 `https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${lon}&appid=${OPENWEATHER_API_KEY}&units=metric`
);
 const data = await response.json();
 return data;
 },
};
```
```

Then in your ``+page.server.ts``:

```
```typescript
import { weatherService } from "$lib/services/weather-service";

export const load = async () => {
 const weather = await weatherService.getWeather(53.2744, -9.0489); // Galway coords
 return { weather };
};
```
```

And in your ``+page.svelte``:

```
```svelte
<script lang="ts">
 const { data } = $props();
</script>

<p>Temperature: {data.weather.main.temp}°C</p>
<p>Description: {data.weather.weather[0].description}</p>
<p>Humidity: {data.weather.main.humidity}%</p>
```
```

This uses the coordinates you already have on your stops, so you can show the weather for each stop location. Want me to show how to integrate it with your stops page?

[user] sorry, first I finished my feature charts
[assistant] ```bash
git add .
git commit -m "feat: add charts"

```
git push origin feature/charts
git flow feature finish charts
git push origin develop
```
```

```
[user] I finished my feature weatherAPI
[assistant] ```bash
git add .
git commit -m "feat: add weather API"
git push origin feature/weatherAPI
git flow feature finish weatherAPI
git push origin develop
```
```

```
[user] i finished my session_management feature
[assistant] ```bash
git add .
git commit -m "feat: add session management"
git push origin feature/session_management
git flow feature finish session_management
git push origin develop
```
```

```
[user] deployment on netlify
[assistant] First, install the Netlify adapter:
```

```
```bash
npm install @sveltejs/adapter-netlify
```
```

```
Update your `svelte.config.js`:
```javascript
import adapter from '@sveltejs/adapter-netlify';
```

```
export default {
  kit: {
    adapter: adapter()
  }
};
```
```

Create a `netlify.toml` in your project root:

```
```toml
[build]
  command = "npm run build"
  publish = ".svelte-kit/output"
```

```
[build.environment]
  NODE_VERSION = "18"
```
```

Then:

1. Go to [netlify.com](https://netlify.com) → Sign up/Login
  2. Click **Add new site** → **Import from Git**
  3. Connect your GitHub repository
  4. Set build command: `npm run build`
  5. Set publish directory: `.svelte-kit/output`
  6. Add your environment variables from `.env` in **Site Settings** → **Environment Variables**
- Then deploy! Netlify will automatically redeploy every time you push to `main`.

[user] should i tick this ?

[assistant] Yes, tick it for sensitive values like:

- `MONGO\_URL`
- `CLERK\_SECRET\_KEY`
- `AUTH0\_SECRET`
- `GITHUB\_SECRET`
- `GOOGLE\_SECRET`
- `AUTH\_SECRET`
- `OPENWEATHER\_API\_KEY`

Don't tick it for public values like:

- `PUBLIC\_AUTH0\_DOMAIN`
- `PUBLIC\_AUTH0\_CLIENT\_ID`
- `PUBLIC\_CLERK\_PUBLISHABLE\_KEY`

The rule is simple – anything that starts with `PUBLIC\_` doesn't need it, everything else should be marked as secret.

[user] how to allow netlify access my private repo in git hub

[assistant] 1. Go to **Netlify** → Add new site → Import from Git → **GitHub**

2. It will ask you to **Authorize Netlify** on GitHub – click Authorize

3. Then it asks which repos to give access to – choose either:

- **All repositories**
- **Only select repositories** → pick your PlaceMark-2.0 repo

If you already authorized Netlify but can't see your repo:

1. Go to **GitHub** → Settings → Applications → **Authorized OAuth Apps**

2. Find Netlify → click it → **Repository access** → add your repo

[user] 10:33:18 PM: build-image version: ac6eb13fbf000e5c09ad677efd8b7c3c2d0142b6 (noble-new-builds)

10:33:18 PM: buildbot version: de14d13d022c18f089944497bd804e269fc4f82f

10:33:18 PM: Fetching cached dependencies

10:33:18 PM: Failed to fetch cache, continuing with build

10:33:18 PM: Starting to prepare the repo for build

10:33:18 PM: No cached dependencies found. Cloning fresh repo

10:33:18 PM: git clone --filter=blob:none https://github.com/villaca13/PlaceMark-2.0

10:33:18 PM: Preparing Git Reference refs/heads/main

10:33:19 PM: Failed during stage 'preparing repo': Unable to access repository. The repository may have been deleted, the branch may not exist, or permissions may have changed: Error checking out submodules: fatal: No url found for submodule path '.claude/worktrees/condescending-cohen-899a83' in .gitmodules : exit status 128: fatal: No url found for submodule path '.claude/worktrees/condescending-cohen-899a83' in .gitmodules : exit status 128

10:33:19 PM: Error checking out submodules: fatal: No url found for submodule path '.claude/worktrees/condescending-cohen-899a83' in .gitmodules : exit status 128

10:33:19 PM: Failing build: Failed to prepare repo

[assistant] The issue is a stale `.claude/worktrees` folder that got committed to your repo. Fix it by removing it:

```
```bash
git rm -r --cached .claude
echo ".claude" >> .gitignore
git add .gitignore
git commit -m "fix: remove .claude worktrees from repo"
git push origin main
```
```

Then trigger a new deploy on Netlify and it should work.

[user] let me finish this deployment feature as well

[assistant] ```bash

```
git add .
git commit -m "feat: add netlify deployment"
```



```
git push origin feature/deployment
git flow feature finish deployment
git push origin develop
git push origin main
git push origin --tags
```

```

10:43:49 PM: 30: {/if}
10:43:49 PM: 31: <slot />;
10:43:49 PM: ^
10:43:49 PM: 32: </div>;
10:43:49 PM: 9:43:49 PM [vite-plugin-svelte] src/routes/stops/+page.svelte:18:25 This reference only captures the initial value of `data`. Did you
mean to reference it inside a derived instead?
10:43:49 PM: https://svelte.dev/e/state_referenced_locally
10:43:49 PM: 16: let latitude = $state(0);
10:43:49 PM: 17: let longitude = $state(0);
10:43:49 PM: 18: let weather = $state(data.weather);
10:43:49 PM: ^
10:43:49 PM: 19:
10:43:49 PM: 20: async function loadWeatherFor(lat: number, lon: number) {
10:43:49 PM: 9:43:49 PM [vite-plugin-svelte] src/routes/user/+page.svelte:10:26 This reference only captures the initial value of `data`. Did you
mean to reference it inside a derived instead?
10:43:49 PM: https://svelte.dev/e/state_referenced_locally
10:43:49 PM: 8:
10:43:49 PM: 9: let { data }: PageProps = $props();
10:43:49 PM: 10: let firstName = $state(data.currentUser.firstName);
10:43:49 PM: ^
10:43:49 PM: 11: let lastName = $state(data.currentUser.lastName);
10:43:49 PM: 12: let email = $state(data.currentUser.email);
10:43:49 PM: 9:43:49 PM [vite-plugin-svelte] src/routes/user/+page.svelte:11:24 This reference only captures the initial value of `data`. Did you
mean to reference it inside a derived instead?
10:43:49 PM: https://svelte.dev/e/state_referenced_locally
10:43:49 PM: 9: let { data }: PageProps = $props();
10:43:49 PM: 10: let firstName = $state(data.currentUser.firstName);
10:43:49 PM: 11: let lastName = $state(data.currentUser.lastName);
10:43:49 PM: ^
10:43:49 PM: 12: let email = $state(data.currentUser.email);
10:43:49 PM: 13: let password = $state("");
10:43:49 PM: 9:43:49 PM [vite-plugin-svelte] src/routes/user/+page.svelte:12:21 This reference only captures the initial value of `data`. Did you
mean to reference it inside a derived instead?
10:43:49 PM: https://svelte.dev/e/state_referenced_locally
10:43:49 PM: 10: let firstName = $state(data.currentUser.firstName);
10:43:49 PM: 11: let lastName = $state(data.currentUser.lastName);
10:43:49 PM: 12: let email = $state(data.currentUser.email);
10:43:49 PM: ^
10:43:49 PM: 13: let password = $state("");
10:43:49 PM: 14:
10:43:49 PM: 9:43:49 PM [vite-plugin-svelte] src/routes/user/[userId]/+page.svelte:13:26 This reference only captures the initial value of `data`.
Did you mean to reference it inside a derived instead?
10:43:49 PM: https://svelte.dev/e/state_referenced_locally
10:43:49 PM: 11:
10:43:49 PM: 12: let { data }: PageProps = $props();
10:43:49 PM: 13: let firstName = $state(data.userIdToBeUpdated.firstName);
10:43:49 PM: ^
10:43:49 PM: 14: let lastName = $state(data.userIdToBeUpdated.lastName);
10:43:49 PM: 15: let email = $state(data.userIdToBeUpdated.email);
10:43:49 PM: 9:43:49 PM [vite-plugin-svelte] src/routes/user/[userId]/+page.svelte:14:24 This reference only captures the initial value of `data`.
Did you mean to reference it inside a derived instead?
10:43:49 PM: https://svelte.dev/e/state_referenced_locally
10:43:49 PM: 12: let { data }: PageProps = $props();
10:43:49 PM: 13: let firstName = $state(data.userIdToBeUpdated.firstName);
10:43:49 PM: 14: let lastName = $state(data.userIdToBeUpdated.lastName);
10:43:49 PM: ^

```

```
10:43:49 PM: 15: let email = $state(data.userIdToBeUpdated.email);
10:43:49 PM: 16: let password = $state("");
10:43:49 PM: 9:43:49 PM [vite-plugin-svelte] src/routes/user/[userId]/+page.svelte:15:21 This reference only captures the initial value of `data`.
Did you mean to reference it inside a derived instead?
10:43:49 PM: https://svelte.dev/e/state_referenced_locally
10:43:49 PM: 13: let firstName = $state(data.userIdToBeUpdated.firstName);
10:43:49 PM: 14: let lastName = $state(data.userIdToBeUpdated.lastName);
10:43:49 PM: 15: let email = $state(data.userIdToBeUpdated.email);
10:43:49 PM: ^
10:43:49 PM: 16: let password = $state("");
10:43:49 PM: 17: subTitle.text = "User Details";
10:43:50 PM: ✓ 214 modules transformed.
10:43:50 PM: ✗ Build failed in 354ms
10:43:50 PM: error during build:
10:43:50 PM: Build failed with 2 errors:
10:43:50 PM: [UNLOADABLE_DEPENDENCY] Error: Could not load src/lib/ui/MenuSplash.svelte
10:43:50 PM: [src/routes/+layout.svelte:4:24]
10:43:50 PM: 4 import MenuSplash from "$lib/ui/MenuSplash.svelte";
10:43:50 PM: No such file or directory (os error 2)
10:43:50 PM: [UNLOADABLE_DEPENDENCY] Error: Could not load src/lib/ui/Menu.svelte
10:43:50 PM: [src/routes/+layout.svelte:3:18]
10:43:50 PM: 3 import Menu from "$lib/ui/Menu.svelte";
10:43:50 PM: No such file or directory (os error 2)
10:43:50 PM: at aggregateBindingErrorsIntoJsError (file:///opt/build/repo/node_modules/rolldown/dist/shared/error-D4ASW9r2.mjs:48:18)
10:43:50 PM: at unwrapBindingResult (file:///opt/build/repo/node_modules/rolldown/dist/shared/error-D4ASW9r2.mjs:18:128)
10:43:50 PM: at #build (file:///opt/build/repo/node_modules/rolldown/dist/shared/rolldown-build-CorVA9v1.mjs:3255:34)
10:43:50 PM: at async buildEnvironment (file:///opt/build/repo/node_modules/vite/dist/node/chunks/node.js:33033:64)
10:43:50 PM: at async Object.build (file:///opt/build/repo/node_modules/vite/dist/node/chunks/node.js:33455:19)
10:43:50 PM: at async Object.buildApp (file:///opt/build/repo/node_modules/vite/dist/node/chunks/node.js:33452:153)
10:43:50 PM: at async CAC.<anonymous> (file:///opt/build/repo/node_modules/vite/dist/node/cli.js:778:3) {
10:43:50 PM: errors: [Getter/Setter]
10:43:50 PM: }
10:43:50 PM: "build.command" failed
10:43:50 PM:
10:43:50 PM: Error message
10:43:50 PM: Command failed with exit code 1: npm run build (https://ntl.fyi/exit-code-1)
10:43:50 PM:
10:43:50 PM: Error location
10:43:50 PM: In build.command from netlify.toml:
10:43:50 PM: npm run build
10:43:50 PM:
10:43:50 PM: Resolved config
10:43:50 PM: build:
10:43:50 PM: command: npm run build
10:43:50 PM: commandOrigin: config
10:43:50 PM: environment:
10:43:50 PM: - AUTH0_ID
10:43:50 PM: - AUTH0_ISSUER
```

```
10:43:50 PM: - AUTH0_SECRET
10:43:50 PM: - AUTH_SECRET
10:43:50 PM: - GITHUB_ID
10:43:50 PM: - GITHUB_SECRET
10:43:50 PM: - GOOGLE_ID
10:43:50 PM: - GOOGLE_SECRET
10:43:50 PM: - MONGO_URL
10:43:50 PM: - OPENWEATHER_API_KEY
10:43:50 PM: - cloudinary_key
10:43:50 PM: - cloudinary_name
10:43:50 PM: - cloudinary_secret
10:43:50 PM: - cookie_name
10:43:50 PM: - cookie_password
10:43:50 PM: publish: /opt/build/repo/build
10:43:50 PM: publishOrigin: config
10:43:50 PM: Build failed due to a user error: Build script returned non-zero exit code: 2
10:43:50 PM: Failing build: Failed to build site
10:43:50 PM: Finished processing build request in 16.379s
[assistant] Two main errors to fix:
```

**\*\*1. Missing files\*\*** – `Menu.svelte` and `MenuSplash.svelte` are not in your repo. Check if they exist locally:

```
```bash
ls src/lib/ui/
```
```

If they exist locally but not on GitHub, they weren't committed. Add and push them:

```
```bash
git add src/lib/ui/
git commit -m "fix: add missing UI components"
git push origin main
```
```

**\*\*2. Deprecated Svelte syntax\*\*** – these are warnings not errors, but fix them anyway. Replace ``<slot />`` with ``{@render children()}`` in ``+layout.svelte``:

```
```svelte
<script lang="ts">
  let { children } = $props();
</script>

{@render children()}
```
```

And replace ``$state(data.something)`` with ``$derived``:

```
```svelte
let firstName = $derived(data.currentUser.firstName);
let lastName = $derived(data.currentUser.lastName);
let email = $derived(data.currentUser.email);
```
```

The missing files are the critical error causing the build to fail. Fix those first.

[user] how to solve it ?

[assistant] Netlify detected secrets in your code/repo. Check if your `.env` file was accidentally committed:

```
```bash
git rm --cached .env
```
```

Make sure `.env` is in your `.gitignore`:

```
```\n.env\n```
```

Then for the Netlify environment variables, make sure these are marked as *****Contains secret values***** in Netlify dashboard:

```
- `cookie_name`\n- `cookie_password`\n- `cloudinary_secret`\n- `cloudinary_name`\n- `cloudinary_key`
```

Then commit and push:

```
```bash\ngit add .gitignore\ngit commit -m "fix: remove .env from repo"\ngit push origin main\n```
```